

面向长程实践的锚点与契约：AC 范式 V6 技术报告（TRAE 特调）

**Anchors and Contracts for Long-Horizon Practice:
AC Paradigm V6 Technical Report (TRAE-Tuned)**

曾庆明（Sammy Zeng）

魏道（Dao Wei）

2026 年

¹联系邮箱: jop.sammy@163.com ORCID: [0009-0007-4382-0040](https://orcid.org/0009-0007-4382-0040)

²本文为杭州市上城区钱江新城中央商务产业社区专项支持项目”长程实践理论”的研究成果，社区在调研对接、基础压力测试样例、真实业务用例方面提供了关键性支持（非经费资助）

Abstract

大语言模型在长程任务（Long-Horizon）中的失败常被归因于单次推理能力不足，但其实质是不可预计算的顺序依赖链与媒介强制导致的结构性断裂。AC 范式（“锚点契约式人机协同工程范式”）正是为此提出的一种协同工程中间层：它将长程实践理论所要求的契约连续性、状态连续性与评测独立性转化为可操作的组织结构，为有限智能体（人与 LLM）协同提供维持长程实践的组织下限。这里所说的“下限”不是替代基座模型能力的总下限，而是在执行基座已具备最低指令遵循、局部推理与基本工具调用能力时，长程连续性仍然额外需要满足的结构下限。本文讨论的不是 AC 范式的全部最终形态，而是其在 TRAE IDE 上的首个完整工程化实例——AC 范式 V6（TRAE 特调版）。我们阐述了基于“无知之幕”的 GN-004 独立审查、对冲性能风险的 subagent 调度等核心机制的设计原理，说明为什么同一套组织原则在落入具体 agent 工具时仍需围绕平台接口做独立调优，并论证了基建噪声对长程实践的系统性影响。最终，通过一道由 DeepSeek-v4-pro 持续 102 小时攻克、历经 16 轮跨断面迭代的组合数学案例，本文展示了 AC 范式的锚点—契约—审查三层结构如何在当前实现环境中支撑长程闭合；这一案例不构成跨平台普适性的充分证明，但构成了该范式在真实 agent 平台上具备工程可行性的首个存在性证据。

关键词：锚点契约范式；长程实践；人机协同；subagent 调度；GN-004
独立审查；结构下限

Contents

1 引言	5
1.1 长程不是时间长	5
1.2 经验观察：三类结构性断裂	5
1.3 本文回答什么	6
2 理论承接：从长程实践到 AC 范式	6
2.1 四种不可预计算来源	6
2.2 三层结构下限	7
3 AC 范式定义	9
3.1 对象：agent 工具环境中的有限智能体协同	9
3.2 抽象层级定位	10
3.3 根问题：长程实践中的结构失配	11
3.4 核心结构与边界	12
4 设计理念	14
4.1 从三层到两层的架构收束	14
4.2 能力锁的四层构件	14
4.3 13 条件编码契约：HC/SC/EC 体系	15
4.3.1 编码的定义与断裂类型	15
4.3.2 为什么要建立这套编码	16
4.3.3 跨组件的联动呼应	16
4.4 Rules 的层次结构	17
4.4.1 组件化分解	17
4.4.2 表达载体的混合策略	17
4.4.3 跨文件的机制联动	18
4.5 Skill 的按需加载策略与渐进式披露	19

4.5.1	渐进式披露与分形同构	19
4.5.2	硬化标准与边界打磨	19
4.5.3	表达载体：显式 Code 与显式路由	19
4.5.4	与长程实践理论的条件对应	20
4.5.5	Skill 字段与 Rules 层的联动设计	20
4.6	GN-004 无知之幕原则	21
4.6.1	无知之幕的由来与工程化	21
4.6.2	三层无知：从 rules 到 Skill 到 GN-004 自身的去身份化	22
4.6.3	三类强制触发与审查密度	23
4.6.4	默认审查逻辑：断裂推论	23
4.6.5	HARD_BLOCK 与 SOFT_BLOCK：审查结论的工程语义	24
4.6.6	人类主动审查：质量面的扩展	24
4.7	锚点体系的分工与弹性扩展	25
4.7.1	七类基础锚点与工程交接	25
4.7.2	锚点闭合与 GN-004 审查联动	26
4.7.3	下限设计：实际业务中的锚点衍生	27
5	TRAE 特调	27
5.1	为什么需要针对性特调	28
5.2	从 skill-creator 到 acp-skill-creator-6-1	29
5.3	部署工具的同源证据	30
5.4	非通用性声明	30
6	Subagent 与媒介强制	31
6.1	注意力浓缩与局部压缩	31
6.2	注意力预算的分层策略	31
6.3	最优性能投影与风险对冲	32
6.4	并行上限的工程含义	34
7	基建噪声与平台约束	35
7.1	基建噪声的结构性影响	35

7.2	102 小时案例中的基建噪声实证	36
7.3	基建噪声不是偶发问题	37
8	102 小时案例：AC 范式下的长程实践	38
8.1	案例概览	38
8.2	为什么它是长程实践	39
8.2.1	不可预计算顺序依赖链：wrapping 的结构性屏障	40
8.2.2	两个关键突破都发生在”不执行就不会知道”的位置	40
8.3	案例时间线	41
8.3.1	第一阶段（0–60h）：穷举路线	41
8.3.2	第二阶段（60–86h）：工程极限压榨	41
8.3.3	第三阶段（86–102h）：代数降维与最终收敛	42
8.4	为什么 AC 结构在该实现中能支撑它	42
8.5	哪些机制在起作用	44
9	局限性与 V7 方向	45
9.1	局限 → V7 方向映射	46
9.1.1	TRAE 特化 → 跨平台泛化	46
9.1.2	禁止 subsubagent → subsubagent 嵌套支持	47
9.1.3	agent id 能力不均 → agent id 统一可追溯	47
9.1.4	系统元信息不可见 → 系统元信息可观测	48
9.1.5	回滚/续接能力缺失 → 状态机快照 + 灵活回滚	48
9.1.6	本地机性能 → 民用机性能优化	49
9.1.7	工具调用配额硬上限 → 工具调用配额弹性扩展	50
9.2	归因与边界	51
10	致谢	51

1 引言

长程实践（long-horizon）领域目前缺少的，不是更会单次作答的模型，而是一层独立可用的协同工程组织。少数工程化靠前的方案与具体平台深度绑定，无法剥离复用；其余表现则更多来自基座模型逐步提升的暴力推理能力，与长程实践的结构性问题并不对应。AC 范式的提出动机正在于此：放弃等待一个抽象的“万能 agent”，转而将长程实践理论^[1]的结构要求逐一转化为可操作的组织原则。本文讨论的 AC 范式 V6 是这一思路在 TRAE IDE 上的首个完整工程化实例，而不是 AC 范式全部最终形态的完成版。这一思路在谱系上承接了早期交互式计算^[2]与增智型人机协同^[3]的传统，但将焦点从单次人机耦合转移到跨断面的组织连续性。

现有的 Benchmark 体系衡量的是单次闭环：得分、完成率、准确率的共同前提是任务信息在起点已经齐备。这恰好是长程任务不满足的条件。

1.1 长程不是时间长

长程实践的核心定义不是时间跨度，而是是否存在不可预计算的顺序依赖链^[1]：步骤 $i + 1$ 的关键输入 K_{i+1} 必须在步骤 i 执行后才能获得，且 K_{i+1} 的取值无法在步骤 i 执行前被预计算。

1.2 经验观察：三类结构性断裂

在本文所考察的 TRAE 实现环境中，DeepSeek-v4-pro 的单次推理已经足够强，但任务拉长到数小时后（如一道 102 小时的组合数学问题），失败仍然主要来自三类结构性断裂而非单次推理不足：

在长程实践中，上下文增大无法代偿未生成的真值——当 K_{i+1} 在步骤 i 执行前尚未生成时，更大的上下文窗口无法填补这一信息缺口。

在连续多天的执行中，对话历史不是可靠的状态存储。隔天恢复上下文时，前一天的推断链可能已被滚动挤出——恢复的是已失序的对话片段，不是完整的推理结构。

在执行者注意力衰减条件下，在“这条路线是否应该继续”的判断点上，执行者倾向于选择继续。不是因为分析支持——是因为“继续”是上下文里

可立即执行的操作，而判断”不该继续”所需的信息不在窗口里。

三种断裂的源头相同：对话历史、上下文窗口、Task 闭合——这些工具协议在设计上假定单次闭环。长程任务的信息结构不服从这个假定。

1.3 本文回答什么

本文以 AC 范式 V6 的 TRAE 实现为主线，但论证对象分为两个层次：第 3 至第 4 节回答 AC 范式作为协同工程组织原理“是什么、为何如此设计”；第 5 至第 9 节回答这一组织原理在 TRAE 上如何被实现、受到了哪些平台约束，以及这些约束将如何导向下一步演进。换言之，本文既不是把 AC 范式等同于一套 TRAE 插件，也不是绕开实现问题空谈抽象原则，而是试图同时说明“范式本体”与“当前实现剖面”之间的关系。第 8 节则以 102 小时组合数学案例展示这套结构在当前实现环境中的实际运转方式与证据边界。

2 理论承接：从长程实践到 AC 范式

AC 范式 V6 不是凭空设计出来的——它是对长程实践理论^[1]的系统性工程回应。该理论的每一项结构判断（三重运载前提、六项必要条件、七类工程补偿）在 V6 中均有对应的设计组件。本节逐项整理这些理论要点，作为后续各节设计说明的理论锚点。

2.1 四种不可预计算来源

长程实践理论从”关键输入为何当前不可得”出发，穷举得到四种互斥且穷尽的来源：

1. **认知缺口（被动）**：不执行到那一步就不知道。在 102 小时案例中，(q,t)-Catalan 等价发现发生在第 76 小时——因为它依赖前 75 小时的路线尝试中积累的 wrapping 结构认知。不执行前面的路线，在我们的实验观察中无法预先推知这个等价。

2. **媒介强制（客观约束）**：执行介质的物理上限迫使任务分段。Context window 上限是最直接的媒介强制——当长程任务的上下文总量超过窗口容量时，信息天然需要压缩、取舍、传递，在现有上下文窗口约束下无法实现”全量保持”。
3. **决策分叉（主动）**：中间选择改变了后续空间。在 102 小时案例中，第 40 小时附近选择代数路线还是组合路线作为主攻方向，直接影响后续数十小时的推理路径——这是一个不可回溯的一刀切决策。
4. **外部漂移（独立）**：外部世界独立变化。在多智能体协作或平台依赖场景中，协同方的需求变更、API 的限流策略调整等外部状态变化独立于执行者发生，无法在任务启动时预知。

在 TRAE 语境下，媒介强制（context window 上限）是最主要的压迫来源，但决策分叉和外部漂移同样压迫价值闭合——增大上下文窗口无法解决这两个来源。

2.2 三层结构下限

长程实践理论建立了三层不可互相替代的结构下限：

- **契约层**：跨检查点稳定存在的不可变约束。在 TRAE 工程组织中对应为 public/目录下的数据 Schema、接口存根 (.pyi)、配置 Schema，以及两核文件中的思想层（”AC 是什么”的判据）。
- **状态层**：跨执行断面传递的当前状态。在 TRAE 工程组织中对应为 spec 三件套、note、.trae/documents/变更记录、三项交接结构（工程过程/交接状态/最终结果）。
- **评测层**：独立于执行者的检查机制。在 TRAE 工程组织中对应为 GN-004 独立审查（无知之幕原则、三类强制触发、三级结论）。

这三层是长程实践理论推导出的”不可删减的下限”。长程实践理论回答的问题是”长程实践需要哪些结构下限”，AC 范式 V6 回答的问题是”在 TRAE 中，如何为这些下限建立工程组织”。AC 范式 V6 的工程组织是围绕

这三层下限的一种可行实现，以下各节逐一展示这种实现的内部逻辑。理论 → 工程 → 平台 → 未来的承接关系见图 1。



Figure 1: 长程实践 → AC 范式 → TRAE 特调 → V7 方向承接图

3 AC 范式定义

AC 范式的全称是“锚点契约式人机协同工程范式”（Anchor-Contract Human-Machine Collaborative Engineering Paradigm）。“AC”取“锚点（Anchor）”与“契约（Contract）”之首字母。锚点体系（第 4 节 § 锚点体系）承担跨执行断面的状态连续性，契约体系（第 4 节 § 三层契约定义）承担跨角色的不变约束，二者共同构成 AC 范式工程组织的基础骨架。需要特别强调的是：AC 范式首先是一个抽象的组织原理，用来把长程实践理论转化为可操作的协同结构；本文中的 V6 则是这一原理在 TRAE 参考系下的首个完整工程化实例，而不是 AC 范式本身的全部边界。

3.1 对象：agent 工具环境中的有限智能体协同

AC 范式的直接对象不是“LLM”或“AI agent”，而是“在 agent 工具环境（如 TRAE IDE）中协同运作的有限智能体”。这里的“有限”同时指人和 LLM——二者在信息获取、推理深度、注意力持续性和可回滚能力上各有不同但同属有限的局限。

V5.3 的工程实践已经验证了一个关键命题：在 TRAE 这种 agent 工具环境中，人和 LLM 通过同一个感知-行动闭环耦合为整体——人通过 IDE 界面、AGENTS.md、spec 文件感知和约束系统行为，LLM 通过工具调用（Read、Write、RunCommand 等）感知和修改文件系统。各自拆开都不构成完整智能体：人缺少工具调用通道，LLM 缺少价值判断权。

因此，AC 范式将人和机器同时视为同一个有限智能体的内部构成元素。人持有价值判断权不意味着“两个智能体”——正如操作系统内核态和用户态权限不同不意味着它们是两个操作系统。这是内部权限分层，不是两个独立智能体的对话。这一立场将认知过程视为分布在人、工具与外部表征构成的耦合系统之中的活动，在认知科学上承接了延展心智^[4]与分布式认知^[5]的理论传统——认知不限于皮肤之内，而是分布在构成工作系统的全部要素之间。

V5.3 的工程实践——AC 范式在 25 个 Skill、三层核文件、500+ 次长程实践中的运行——提供了直接证据：

- 当人类通过 AGENTS.md 注入的约束与 LLM 的工具调用行为在同一

闭环中一致响应时，系统表现出稳定的长程连续性；

- 当这个闭环断裂（人类未及时更新锚点文档、LLM 未读取最新契约）时，系统立即出现状态漂移——且漂移的方向和幅度与闭环断裂的位置一致。

3.2 抽象层级定位

AC 范式的抽象层级是独立的：它继承智能基础理论的底层机理，复用现有 Agent 工具的运行时基建，独占解决长程实践的连续性约束问题。

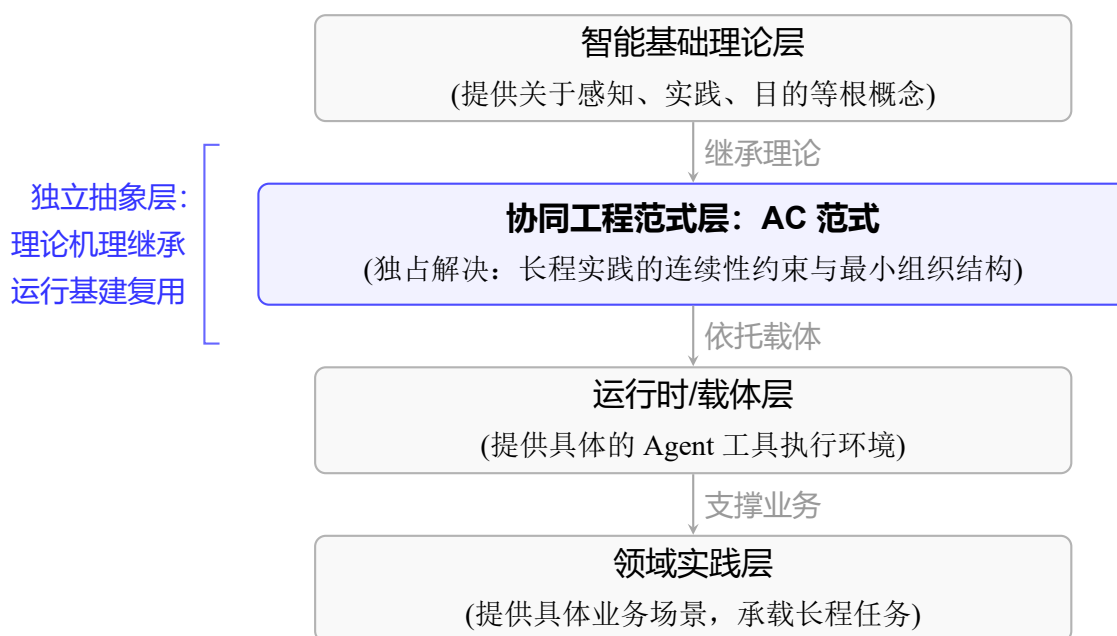


Figure 2: AC 范式抽象层级定位图

- **智能基础理论层**：提供关于“智能体交互”与“有限理性”的底层解释（如 Simon 的有限理性理论^[6]、Observer-to-Agent 统一框架^[7]），以及长程实践理论^[1]对“不可预计算顺序依赖链”与结构性失效的系统分析。AC 范式不研究智能体本质，而是直接继承这些理论基础。

- **协同工程范式层（AC 范式）**：独占解决“长程协同的最小组织结构”问题。提供跨越多个执行断面的连续性约束。
- **运行时/载体层**：提供具体的 Agent 工具执行环境。AC 范式不开发独立的客户端或 IDE，必须依托现有 Agent 工具作为载体。
- **领域实践层**：具体的业务场景。AC 范式不包含特定领域的业务逻辑，而是为这些领域的长程任务提供通用的协同框架。

这种层级划分澄清了 AC 范式 V6 的存在形态。尽管 V6 在物理载体上表现为一组针对 TRAE 深度特调的组件，但其在抽象面上并不等同于“一个 TRAE 插件”或“特定工具的操作 SOP”。它回答的是独立于任何特定平台的核心问题：当有限智能体的实践无法由单次闭环完成时，必须具备哪些连续性结构。这里还需进一步区分两件事：一是**范式的抽象独立性**，即 AC 所回答的问题不依附于某个特定 IDE；二是**实现的可迁移性**，即同一套组织原则在不同 agent 工具上仍需围绕其工具协议、调度接口与交互载体做独立实现。V6 当前完成的是后一问题在 TRAE 参考系下的一次完整实例化，而不是全部平台上的一次性通解。

3.3 根问题：长程实践中的结构失配

AC 范式定义的根问题是：长程实践持续推进需要哪些最小结构。

这是一个下限问题，不是工程优化问题。这里所说的“下限”是长程连续性的组织下限，而不是对任意低能力基座都自动生效的能力下限。AC 范式不替代基座在单步指令遵循、局部推理与基本工具调用上的最低执行能力；相反，它假定这些最低能力已经存在，并进一步回答：在此基础上，若任务跨越多个执行断面，系统至少还需要哪些额外组织结构才不会塌陷。长程实践可维持的设计至少需覆盖以下三类约束：

1. **契约连续性**：跨检查点的不可变约束不能被执行过程的便利性侵蚀。没有这一层，每次上下文截断后，执行者无法判断“什么是对的”。
2. **状态连续性**：跨执行断面的当前状态必须显式传递，不能依赖执行者记忆或对话历史。没有这一层，每次恢复上下文后，执行者不知道“做到了、为什么、接下来干什么”。

3. **评测独立性**：对前两者的检查必须独立于执行者。没有这一层，执行者自审存在利益冲突——即“既是运动员又是裁判员”的结构性问题。

这三条共同构成 AC 范式的结构边界：任意一个宣称支持长程实践的方案，其设计必须覆盖这三类约束。

3.4 核心结构与边界

从抽象层看，AC 范式的核心结构由以下组织部件构成（部件间的详细关系见第 4 节）：

- **契约层**：跨检查点稳定存在的不可变约束，用于回答“什么不能被执行便利性侵蚀”。
- **状态层**：跨执行断面传递的当前状态，用于回答“做到哪了、为什么、接下来从哪继续”。
- **评测层**：独立于执行者的检查机制，用于回答“谁来独立验证前两层没有漂移”。
- **锚点体系**：将契约与状态外置为稳定表征，承担跨断面的读取、交接与留痕职能。
- **调度与执行机制**：将局部任务隔离、顺序关系控制、人机信号转型与独立审查编排为可持续实践的操作流。

在本文讨论的 V6/TRAE 实现中，上述抽象部件被进一步映射为一组具体工程构件：两核文件（思想层与能力锁）、8 个 Rules 文件、11 个 Skill、GN-004 独立审查、7 类锚点体系，以及独立配套组件如 acp-skill-creator-6-1、需求结构化分析 Pipeline 与 deploy-ac-v6-components。它们描述的是 AC 范式在 TRAE 参考系下“如何被实现”，而不是 AC 范式在定义上“只能如此存在”。



Figure 3: 组件全景图

AC 范式的边界是明确的：它不回答“LLM 如何推理”（那是基座模型的事），不回答“特定任务如何执行”（那是领域实践的事），不回答“IDE 如何设计”（那是平台的事）。它只回答一个窄但不可替代的问题：当有限智能体的实践无法由单次闭合完成时，需要怎样的组织约束使持续推进成为可能。

4 设计理念

第3节定义了AC范式是什么。本节展示各组织部件的设计决策：结构理由、备选方案排除的原因、与第2节理论承接的推导关系。

4.1 从三层到两层的架构收束

核文件曾采用三层结构，但在持续工程中暴露了规则库层作为独立层的必要性不足：当能力锁（机制层）建成后，规则库的全部内容已被能力锁的三层结构（元约束/rules 域约束/Skill 执行）吸收。因此 V6 将其收束为两层：‘AC 范式思想_v6.md’（判据层：AC 是什么）和 ‘AC 范式能力锁_vTRAE_v6.md’（机制层：如何把判据组织成持续实践机制）。这不是”简化”，而是”识别冗余并拆除”。

4.2 能力锁的四层构件

能力锁内部按抽象层次分为四层：

1. **元约束层（rules-0）**：跨项目稳定存在的不变行为规则。承载全部 13 条条件（HC-1~3 / SC-1~3 / EC-1~7）的直接约束落点。元约束不是”最详细的规则”而是”最不依赖具体场景的规则”——无论项目内容是什么，这些规则都成立。
2. **域约束层（rules-1~7）**：将元约束落入具体场景。例如 rules-1（阶段识别）将 SC-2（顺序约束）落入 8 阶段 pipeline，rules-5（锚点体系）将 HC-3（状态传递）落入七类锚点的具体维护规则。
3. **执行层（11 Skill）**：以 Skill 为载体定义”用什么工具、按什么顺序、在什么触发条件下”执行具体操作。Skill 不承载规则判断——规则由 rules 层承载，Skill 只承载”给定规则已知的前提下，如何执行”。
4. **独立审查层（GN-004）**：独立于前三层的验证机制。GN-004 不是执行者的升级版——它不执行任务，不纠正代码，不替代前三层。它只做一件事：按预定义 rubric 独立验证前三层的产出是否合规。

这四层的设计遵循一个原则：每层对上一层的产出只做验证，不做修改；对下一层的产出只提供约束，不替代执行。

4.3 13 条件编码契约：HC/SC/EC 体系

在上述四层构件中，长程实践理论提出的“六条逻辑必要条件”与“七条工程补偿条件”被系统性地编码为 13 个缩写标识（HC-1~3、SC-1~3、EC-1~7）。这一编码体系是贯穿 V6 核心设计的关键语言。

4.3.1 编码的定义与断裂类型

这套编码根据条件缺失后引发的“断裂类型”对逻辑必要条件进行了严格的硬/软划分，并将工程补偿独立归类：

- **HC (Hard Conditions, 硬断裂条件)**：包含 HC-1（目的闭合）、HC-2（检查点分解）、HC-3（状态传递）。这三条构成了长程实践的物理与逻辑下限。当其缺失时，执行会发生“硬断裂”——系统不知道要做什么、不知道怎么拆解、或者交接时信息完全蒸发，导致实践在物理或逻辑上直接无法继续推进。
- **SC (Soft Conditions, 软断裂条件)**：包含 SC-1（价值闭合）、SC-2（顺序约束）、SC-3（未闭合标记）。这三条约束了实践的有效性。当其缺失时，执行会发生“软断裂”——系统表面上仍在忙碌推进（例如将错误分支标记为已完成并继续），但整条依赖链已建立在假闭合之上，最终结果在结构上不可靠。
- **EC (Engineering Compensations, 工程补偿条件)**：包含 EC-1~7（如恢复回滚、隔离边界、独立评测、机向人解释与请示等）。这是为了应对真实环境中的执行偏航、能力缺失、意图断裂等高概率噪声而设置的装甲层，确保前两类逻辑条件在充满噪声的基座环境中能够真正落地。

4.3.2 为什么要建立这套编码

设计这套编码契约绝非仅为缩写便利，而是为整个 AC 范式建立了一套跨组件的“类型学语言”。

长程实践的复杂性要求多个维度的机制相互配合。如果没有统一的编码，一个阶段的失败很难被准确定性，各组件间的联动也会沦为模糊的自然语言描述。通过引入 HC/SC/EC 的前缀，每个条件都自带了对其失效后果严重度与应对策略的声明：一旦触及 HC 失效，意味着出现了根本性的物理或逻辑阻断；一旦触及 SC 失效，系统就必须引入人类价值判断或独立评测去证伪假闭合；一旦触及 EC 失效，则需要启动错误隔离或向人请示等补偿机制。

4.3.3 跨组件的联动呼应

这套编码契约成为串联整个 V6 组件矩阵的血脉，后续各核心组件均围绕其展开结构联动：

- **在 Rules 层中划定边界：**元约束（rules-0）不再是平铺直叙的条例，而是直接以这 13 个编码为核心索引建立约束条目。这让执行者清晰知道哪些是绝对不可妥协的硬底线（HC），哪些是需要人机对齐的软判断（SC/EC）。
- **在 Skill 矩阵中显式声明：**每个 SKILL.md 文件的首个必填字段即为 hc_sc_ec_mapping。Skill 在设计阶段就必须显式声明本阶段的操作流承载了哪些具体条件，以此确保从理论到实践映射没有结构性遗漏。
- **在 GN-004 中量化审查：**独立审查机制（GN-004）的 Rubric 评分标准直接与这些编码挂钩。GN-004 不依赖“看起来好不好”的模糊感觉，而是精确测量当前产出是否击穿了 HC-3（状态是否可靠传递）或 SC-3（是否如实打上了未闭合标记）。条件映射将审查从主观意见转化为了可复现的结构测量。

4.4 Rules 的层次结构

4.4.1 组件化分解

V6 的 8 个 rules 文件（rules-0~7）不是对旧有规则库的简单移植。过去实践中，规则库以长篇教学文档形态独立存在，甚至曾尝试将大量内容压入单一的 rules 文件中，但这不仅违背了长程实践的维护性原则，更造成了沉重的上下文负担。V6 贯彻 AC 范式的组件化思想，将其拆解为能力锁内部的域约束构件——每个 rules 文件作为独立的功能约束，可被 TRAE 的‘alwaysApply’机制独立注入而互不污染，从而使得组件易维护、可维护。

4.4.2 表达载体的混合策略

在规则的表达载体上，多轮实践证明纯自然语言与纯机械语法均有不可调和的局限性。V6 最终采用”中文为主、中英混合、Python 语法演绎逻辑、其他程序性语法辅助”的混合策略，这是在我们多轮工程实践中被验证为最有效的策略：

- **中英混合策略的权衡：**采用中文作为主体，核心在于其信息密度高，在保持意图准确的前提下能压缩 30%~45% 的字符数，有效降低上下文负担。但不能采用纯中文设计的根本原因在于：在 coding、action、function call 过程中必然涉及英文代码，在长上下文的高压注意力环境下，强行翻译或脱离英文语境会导致这三者的执行性能显著下降。
- **机械语法与自然语言的边界：**采用 Python 等机械语法演绎核心逻辑（如 ec7_action_gate），能有效消除自然语言歧义、提高关键描述准确性，并在复杂逻辑下大幅降低字符数。然而，避免采用纯机械语法的核心考量是：某些语义级问题（如价值判断）若被过度形式化，会变得过于死板，使执行者丧失”智慧的灵活性”。因此，V6 在明确的约束边界内保留了自然语言，给予 Agent 必要的推断空间。

图 4 展示了 8 个 rules 文件的职责分工、交叉引用关系以及与长程实践理论条件的承载关系：

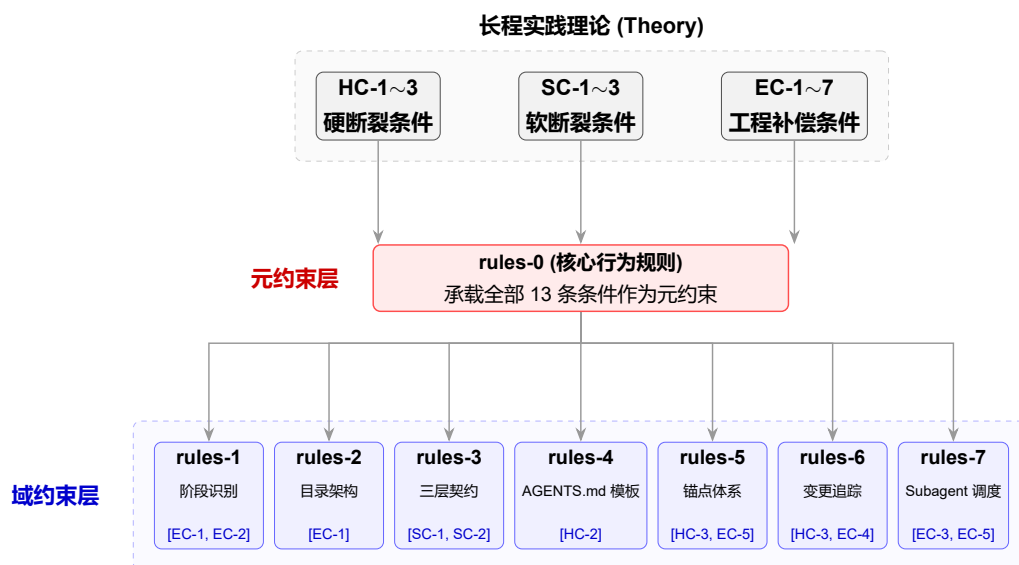


Figure 4: Rules 层结构：元约束与域约束对长程实践理论条件的承载关系

4.4.3 跨文件的机制联动

EC-3（独立评测）在 6 个 rules 中交叉落点——rules-0（GN-004 触发机制）、rules-2（public/契约保护）、rules-3（契约测试套件）、rules-5（锚点闭合校验）、rules-6（文档完整性审查）、rules-7（subagent 隔离审查）——覆盖面最广，反映独立评测是 AC 范式最核心的组织约束。这不仅是在不同文件中重复同一句话，而是将“独立评测”的抽象条件转化为各领域的具体执行检查点。例如，在 rules-2 中，评测落点为“公共契约区是否被违规篡改”；在 rules-5 中，评测落点为“上下游锚点是否形成逻辑闭环”。

跨文件保护机制采用三重防火墙：rules-0 § 四-10（public/目录保护）+ rules-4 §4.3（禁止操作 public/）+ rules-0 § 四-7.2（ec7_action_gate 工具调用拦截）。多层防御防止任何一层被绕过。这种设计源于压测中发现的单点失效风险：仅靠单一文本规则（rules-4）声明“禁止修改”，在执行者面临明确指令（如“清理无用文件”）时可能被忽略。因此，V6 引入了双重文本覆盖（rules-0 与 rules-4 相互印证增强权重），并在工具调用路径（DeleteFile/Write）上设置了强制的 Python 伪代码拦截层（ec7_action_gate），将“规则声明”升级为“操作前闸门”。

4.5 Skill 的按需加载策略与渐进式披露

4.5.1 渐进式披露与分形同构

早期设计中，Skill 采用全部常驻策略——当行为空间封闭且已知时，穷举等价于完备。V6 将行为空间扩展到”长程实践”（开放空间）后，穷举策略失效：因为可能的长程实践形态是无限的，而上下文预算是固定的。

V6 的 Skill 系统因此采用”渐进式披露”设计，其核心不是”frontmatter 常驻 → 正文按需加载”（name 和 description 本来就在 TRAE 的 Skill 体系中始终可见），而是 Skill 作为”闸门 + 能力”的双重角色的模块化分解。

这一设计的底层逻辑来自长程实践理论的一个核心推论：构筑与执行是分形同构的。一个实践阶段的结构设计（构筑）与该阶段的执行操作（执行）共享同一套抽象——如果为构筑和执行分别设计独立系统，就会在边界处产生不可避免的语义漂移和契约不一致。因此 V6 将二者统一收纳到同一个 Skill 体系中：Skill 既定义了”这个阶段要完成什么结构”（构筑），也定义了”用什么操作序列完成它”（执行）。

4.5.2 硬化标准与边界打磨

在硬化标准上，11 个 Skill 收束为长程实践中经过多轮验证的最小且值得硬化的行为抽象集。过多设计会带来上下文负担，过少则无法覆盖长程实践理论的结构下限。更重要的是，每个 Skill 与 rules 层之间的边界经过了精细打磨——如果 Skill 和 rules 在同一行为上给出了结构性矛盾的指令，Agent 在高压注意力下的执行性能会显著下降。因此，Skill 不重复 rules 已覆盖的声明式约束，只硬化 rules 无法有效约束的”操作工序”。同时，由于构筑与执行分形同构，Skill 所选择的抽象必须在抽象层上同时有效映射”构筑”与”执行”两个维度——如果抽象仅在一侧充分而在另一侧薄弱，行为就会在薄弱侧发生断裂：例如一个 Skill 产出了完善的结构设计但缺少对应的执行指引，执行者就会自行填补操作空白，从而偏离设计意图。

4.5.3 表达载体：显式 Code 与显式路由

在表达载体上，实践表明 Skill 不应依赖语义描述让 Agent 基座去”悟”该做什么——工具调用的 action 代码（如 AskUserQuestion、NotifyUser、

TodoWrite 等 TRAE 专用工具的具体使用方式) 必须显式写入 Skill 正文。更进一步, 当流程或行为间存在显著顺序关系时 (如 s0101→s0102→s0103), Skill 应显式路由到下游 Skill, 而不是让 Agent 自行判断“下一步该干什么”。实践验证了缺失显式路由的风险: 当“聪明的”模型在完成一个阶段后主动执行“他觉得下一步要做什么”, 程序正义即告失败——Agent 跳过了路由逻辑, 进而跳过前置条件检查和触发条件匹配, 由此产生锚点偏移和契约漂移。从长程实践的多节点视角看, 漂移不是偶发 bug——在缺失显式路由的条件下, 它是在多节点长程实践中反复出现的系统性模式。

4.5.4 与长程实践理论的条件对应

这一设计与长程实践理论的条件体系直接对应: HC-3 (状态传递与交接) 在 Skill 流水线中落为每个 Skill 的闭合信号与交接协议, 使得每个节点都是显式状态转移边界——阻断漂移在状态层的传播; SC-2 (顺序约束) 通过触发条件和前置条件天然实现阶段编排, 而显式路由机制进一步确保即使 Agent 具备推断能力, 也不得跳过或倒序执行; HC-1 (目的闭合) 与 HC-2 (检查点分解) 要求每一步的产出必须符合预定义的闭合判据——每个 Skill 的闭合判据定义了“允许声称闭合”的最低下限, 而构筑-执行的分形同构确保了闭合判据在结构和操作两个维度的自洽性: 结构上该完成的不会在操作上被遗漏, 反之亦然。

4.5.5 Skill 字段与 Rules 层的联动设计

Skill 与 rules 层的联动关系体现在每个 SKILL.md 的字段设计中。每个字段不是孤立的功能描述, 而是能力锁中某个约束在具体执行面上的落点:

- hc_sc_ec_mapping: 声明当前 Skill 具体覆盖了哪些条件 (如 HC-3/SC-2/SC-3), 使 GN-004 能独立核对“Skill 声称覆盖的条件是否真的在 Action Flow 中得到了执行”——这是 EC-3 (独立评测) 从 rules-0 到 Skill 的串联通路。
- State Protocol: 将 rules-0 中“状态同步与生成规范”的抽象约束转化为三值状态 (已闭合/未闭合/当前不可判定), 使执行者的进度在任何时刻可被外部独立读取。

- `ec7_trigger_gate` 与 `ec6_minimum_explanation`: 将 `rules-0` 中 EC-7（开放判断转型）和 EC-6（意图传达）的行为转型机制落地为 Skill 专属的”在什么条件下必须触发人机交互”和”交互时必须向人类提供哪些最低信息”。
- `gn004_touchpoints`: 在 Skill 设计时便预设审查触点——在不同阶段（三件套/核心段/执行中/交付前）GN-004 应检查什么。这使得独立审查不是事后补救，而是 Skill 设计阶段就已嵌入的结构性组件。

这种字段设计将 `rules` 层的声明式约束（”必须做 X”）转化为 Skill 层的操作性约束（”在做 X 时，以什么格式、在什么时机、向谁汇报”）。`rules-0` 声明合法性条件，Skill 定义合法性条件的落地工序——二者分工但不重复。

4.6 GN-004 无知之幕原则

GN-004 是 AC 范式中 EC-3（独立评测）的工程载体。其设计围绕一条核心原则——”无知之幕”——展开，在工程实践中被验证为保障独立审查起点公平的关键机制。

4.6.1 无知之幕的由来与工程化

”无知之幕”（Veil of Ignorance）源自政治哲学家罗尔斯（John Rawls）在《正义论》^[8]中提出的思想实验：当人们设计社会的基本规则时，需要设想自己身处一道”幕布”之后——幕布遮去了每个人具体的社会身份、阶级地位、天赋资源，使得决策者完全不知道自己将在设计出的社会结构中处于何种位置。正因为无法预知自己的身份，决策者才会倾向于设计出一套对所有人都公平、不论落在哪个位置都不会吃亏的规则。

将这一原则从社会契约迁移到工程审查时，其有效性来自完全相同的机制：审查者在不知道被审查者身份的情况下，无法对不同来源的产出施加不同的审查标准。在实践中得到验证——当 GN-004 预知执行者身份后，审查质量会向两个方向退化：若知道”这是主线程在执行”，审查标准可能被放宽（”同一上下文、不会太离谱”的隐含预设）；若知道”这是某个 `subagent` 在执行”，可能预设该 `subagent` 的产出不可靠并过度审查，造成不必要的返

工。去除身份信息后，每一份产出只按其内容被独立评判——审查结论不再是对“谁做的”的预设回应，而是对“做成了什么样”的结构测量。

4.6.2 三层无知：从 rules 到 Skill 到 GN-004 自身的去身份化

无知之幕并非仅在 GN-004 被拉起的那一刻生效，而是贯穿三个层面的系统性“无知”设计。

第一层——rules 层与 Skill 层对 GN-004 的内部无知。rules 只描述 GN-004 的外部行为接口（拉起方式、信号类型、响应规则），不暴露 GN-004 内部的审查方法论。Skill 同样如此：每个 Skill 的 `gn004_touchpoints` 字段仅声明“在什么时机应拉起 GN-004 审查什么内容的声明”，但不得包含“GN-004 将如何审查”的任何细节。这确保了执行者不能通过 rules 或 Skill 预判“该被审成什么样”，也就无法在产出中预先规避审查视角；审查者不能通过 rules 层预判“该怎么审”，其审查行为仅由自身的 rubric 驱动。这一层的设计结果是：即使执行者明确知道 GN-004 的审查依据来自长程实践理论的 13 条件体系，执行者在 rules 和 Skill 中也找不到任何暴露 GN-004 内部审查方法论的显式表述——执行者仅知道“审什么”，不知道“怎么审”。

第二层——GN-004 对自身命名的无知。GN-004 内部不知道自己的外部标识叫“GN-004”。这一标识对 rules 层和人类是外部引用标签，但 GN-004 自身只以功能角色被调用：它知道自己的审查方法论（五诊断维度、Rubric 评分标准、13 条件断裂核查），知道自己在长程实践流程中承担的是“独立验证”的角色职责，但不存在一个自我标识叫“我是 GN-004”。这一设计防止了审查者的身份认同固化：审查行为不会因为“我是 GN-004 所以我必须严格”或“我是 GN-004 所以我必须找出问题”而发生系统性偏航——它只是忠实地执行审查方法论，不附加角色标签带来的行为畸变。

第三层——GN-004 对被审查者身份的无知。GN-004 被拉起时不预知执行者的身份，只通过功能描述理解职责。这防止了前述的审查标准双向退化问题。

三层“无知”合在一起构成了一道完整的去身份化屏障：执行者不知道 GN-004 怎么审（第一层），GN-004 不知道自己叫什么（第二层），GN-004 不知道被审查者是谁（第三层）。每一方都被隔离在自己的认知边界内，只能基于产出内容而非产出者身份做出或接受判断。

4.6.3 三类强制触发与审查密度

GN-004 的三类强制触发（T1/T2/T3）对应长程实践理论的三个结构薄弱点——这些薄弱点恰是 13 条件中 HC/SC 硬软断裂最可能发生的节点：

- T1——计划审查：在代码产出前，验证 spec/plan/tasks 是否自洽。阻挡方向偏差进入实施。其主要诊断靶点是 HC-1（目的闭合是否真实）和 SC-2（顺序约束是否有缺失或跳跃）。
- T2——关键检查点审查：在决策分叉/价值判断/漂移检测节点，验证当前路径是否偏离 spec/plan。其主要诊断靶点是 HC-3（状态传递是否完整）和 SC-3（未闭合节点是否被如实标记）。
- T3——交付前闭合审查：在合流前，验证全部产出是否满足预定义闭合信号。这是对所有 HC/SC 条件的全量终检——任何在前置阶段未被捕获的断裂都会在 T3 中暴露。

在审查密度上，即使是一个标准的、结构较为简单的长程实践，也至少包含三次 GN-004 拉起（T1/T2/T3 各一次）。而在真实复杂任务中——例如涉及多 Skill 流水线、多 subagent 并行、或跨决策分叉的长程实践——GN-004 的实际拉起次数远大于 3 次：仅仅在 T2 阶段，每遭遇一次关键决策分叉、每出现一次假闭合的可疑信号，都可能触发独立的 GN-004 审查。T1 和 T3 虽然名义上各一次，但若 T1 发现 HARD_BLOCK 级问题，修正后必须重审；若 T3 发现残留断裂，也会追溯触发分段复审。十位数乃至数十次的 GN-004 拉起在复杂长程实践中是常态而非异常。

4.6.4 默认审查逻辑：断裂推论

GN-004 在默认状态下的审查逻辑并非泛化的“合规性审查”，而是精准对准长程实践理论的断裂推论：GN-004 的核心工作是系统性地逐一检查当前产出是否触发了 HC/SC 中任何一条的断裂条件。如果 HC-1（目的闭合）未被满足——即当前产出没有明确的可核实闭合信号——GN-004 不会打一个“模糊”评语后放行，而是直接返回 HARD_BLOCK。同理，如果 SC-3（未闭合标记）未如实声明——即执行者某子任务未完成但未被标记为“未闭合”——GN-004 返回的也是 HARD_BLOCK 而非“观察项”。

这是 GN-004 与一般 linter 或静态分析工具的本质区别：它不检查格式、语法或风格的合规性，而是检查”这个实践是否会在下一步塌掉”。断裂推论意味着审查结论直接与 13 条件的失效后果绑定——HARD_BLOCK 对应硬断裂条件（HC 失效）或假闭合证据（SC 失效且不可绕过），SOFT_BLOCK 对应需要人类判断的软断裂边界（方向显著偏离但人类可能已知悉并接受），通过意味着当前断面上没有可检出的断裂风险。

4.6.5 HARD_BLOCK 与 SOFT_BLOCK：审查结论的工程语义

GN-004 的三级审查结论直接映射为两种明确的工程阻断信号——HARD_BLOCK 与 SOFT_BLOCK——加上”通过”构成完整的三态结论：

- **HARD_BLOCK（硬阻断）**：当检测到 HC 级断裂或明确假闭合证据时触发。执行者必须在阻断位置修正，修正后重新经过完整独立审查（不得仅审”上次那一点”）。HARD_BLOCK 是最高等级阻断——这意味着如果不修正，后续执行将建立在已断裂的依赖链之上，全量产出最终不可信。
- **SOFT_BLOCK（软阻断）**：当发现方向显著偏离、假闭合证据或批量模板化确认时触发。此类阻断不得由执行者自行绕过——必须送达人类裁定。人类选择”已知悉继续”后才放行。SOFT_BLOCK 的设计意图是在”已知的软断裂风险”与”保持流程推进”之间建立一条人类仲裁通道——阻断不替代人的判断，而是确保人知情。
- **警示放行**：当前无断裂证据，但存在值得关注的观察项。记录观察项后继续执行。观察项在后续检查点中被累积跟踪——如果在后续 T2 或 T3 中这些观察项仍未消除或恶化，可能升级为 SOFT_BLOCK。
- **通过**：无任何阻断或警示项。当前断面上所有 HC/SC 条件均无断裂信号。

4.6.6 人类主动审查：质量面的扩展

前述的 T1/T2/T3 触发均为系统自动审查——GN-004 被 rules 层的触发条件（如阶段完成信号、决策分叉事件）自动拉起，执行断裂推论。但实践

中发现仅靠自动审查是不充分的：长程实践中存在一类质量问题——不是“会不会塌”而是“够不够好”——自动审查的结构测量无法覆盖。例如：产出确实满足了所有 HC/SC 条件（不会断裂），但产出的抽象选择、命名策略、或架构风格可能为后续维护带来额外成本；中文表达的自然语言质量、术语一致性、或论述的严密性同样超出结构测量的覆盖范围。

因此，AC 范式支持人类在任意时刻显式拉起 GN-004，并明确要求审查特定质量面。人类可以在 TRAE 中直接指示“让 GN-004 专门审查当前产出的中文表达质量”或“让 GN-004 审查当前架构抽象是否过度设计”——此时 GN-004 将其常规的断裂推论审查与人类指定的质量维度叠加执行。人类的显式质量指令不替代断裂推论（断裂推论始终是 GN-004 的默认基线），而是在基线之上追加了人类关心的特定维度。

五诊断维度（契约/状态/执行/评测/恢复）同时服务于这两类审查。在自动审查模式中，它们是审查者检查断裂推论的认知框架——13 条件是“被审查对象应满足什么”，五诊断维度是“审查者从哪些角度去看被审查对象”。在人类主动审查模式中，人类可以将质量指令锚定到这五个维度中的某一个或某几个，实现精度更高的定向审查。

4.7 锚点体系的分工与弹性扩展

锚点不是附属文档，而是有限智能体跨执行断面维持连续性的外置约束。从认知科学视角看，锚点文件实质上承担了“物质锚”^[9]的功能——将可变的推理状态绑定到稳定的外部表征上，使跨检查点的认知连续性不依赖于执行者的内部记忆。V6 定义了七类核心锚点，但它们构筑的仅是维持长程实践的下限，而非固化的上限。

4.7.1 七类基础锚点与工程交接

在默认状态下，AC 范式依赖七类基础锚点进行状态管理与认知对齐：

承担工程交接职能的三个锚点（spec 三件套、note、.trae/documents/）是执行者与 GN-004 之间传递上下文的核心载体。它们必须显式包含三段信息：(1) 工程过程——已完成哪些 task/check；(2) 交接状态——当前处于哪个 task/check（三值：已闭合/未闭合/当前不可判定）；(3) 最终结果——已完成部分的验证结论和产出物清单。

Table 1: 锚点体系七类分工表

锚点	定位	工程交接	更新时机
README	用户向入口	否	项目结构/用途变更
report_*	判断沉淀	否	阶段收束时
AGENTS.md	AI 最高优先规则	否	能力锁升级时
spec 三件套	任务级约束	是	任务开始前/阶段切换前
note	跨断面状态传递	是	每个 Task/阶段推进时
.trae/documents/	变更留痕	是	每次变更时
三层契约	公共真相源	否	契约变更时

在这三个交接锚点中，**note** 与 **.trae/documents/** 有着严格的分工边界：前者负责跨断面的状态接力，后者负责逐次变更的留痕与修正归档。两者可以相互引用，但绝对不得相互替代。**note** 的写作元原则是功能性约束而非格式约束：每次写入 **note** 必须包含”做到哪了/为什么/未闭合项/接续入口”四个字段。认知层分区（阅读记录/诊断草稿/审查记录/终态处理）有效防止了不同层级内容的混杂。

4.7.2 锚点闭合与 GN-004 审查联动

锚点不仅是执行者写给下一个执行者的便签，更是 GN-004 进行独立审查的法理依据。在关键检查点和交付前，GN-004 被强制要求独立读取当前活跃的 **spec** 三件套、**.trae/documents/** 下的全部变更记录以及当前 **note**。

GN-004 依赖这些锚点文档中的三段交接信息验证长程实践的闭合状态：过程是否自治、状态是否进行了三值标记、结果是否有可独立验证的证据链。如果执行者试图缩窄读取范围（例如通过对话摘要向 GN-004 隐瞒历史记录），GN-004 将直接标注”试图限制独立读取范围”并触发 AskUserQues-

tion 级别的人类干预。交付前的闭合校验必须确保所有的未闭合项都已被显式标记，而非在长程执行中被沉默遗忘。

4.7.3 下限设计：实际业务中的锚点衍生

必须强调的是，AC 范式固定的这七类锚点实际上是长程实践的**结构下限**，而非实际业务中的**最佳实践上限**。AC 范式在 V6 中并没有彻底写死“锚点文件只能是这些”，正相反，范式提供了充分的灵活度，允许在实际业务层面灵活加入合理的锚点文件以增强 Agent 能力。

在高度复杂的任务中，根据任务特性的不同，执行者可能在计划阶段（T1）额外衍生出特定的锚点文件。例如：

- **调研与拆解锚点：**当需要将一个非常庞大的 SOP 体系拆解为 N 个精简的稽查路线时，单靠一个 note 文件无法承载极高的认知负荷。此时可能需要分层次建立大量的前置调研笔记，以固定第一阶段的结论，这些额外笔记就自然在计划阶段被固化为新的业务锚点文件。
- **审查行为锚点：**在极端复杂的系统重构中，GN-004 自身的高频审查（数十次阻断与放行）也会产生大量的决策上下文。此时，可能需要独立对 GN-004 的历次行为结论建立专项的审查锚点文件（如 GN004_audit_trail.md），作为追踪“系统是如何走到当前状态”的真相源。

这种弹性扩展设计表明，锚点体系不是封闭的教条。只要新的锚点文件能够遵循“过程、状态、结果”的三段交接原则，并能被后续的执行者与 GN-004 有效读取，它们就应该被大胆引入到长程实践中，以抵御复杂任务带来的状态塌陷风险。

5 TRAE 特调

本节讨论的不是“AC 范式等于 TRAE”，而是 AC 范式 V6 在 TRAE 上的特调实现。AC 范式作为抽象组织原理，回答的是长程实践需要哪些连续性结构；而一旦这些结构落入具体的 agent IDE，它们就必须围绕该平台的工具协议、交互载体与调度边界做独立实现。V6 的能力锁命名（vTRAE_v6）、

部署路径（.trae/）和工具调用协议都因此内嵌了 TRAE 的平台假设。这不是对 AC 一般性的收回，而是对“抽象原理如何落地”为何必须特调的一次正面展开。

5.1 为什么需要针对性特调

不同 agent IDE 与基座模型的组合，在最低指令遵循、注意力预算衰减、工具调用行为等基本参数上存在显著差异。SWE-agent 的研究已从 Agent-Computer Interface（ACI）的角度独立论证了工具接口设计对 agent 表现的显著影响^[10]——为特定平台精心调校的接口可以大幅提升 agent 的可靠性，而抽象的通用接口在实践中表现不稳定。在 V5.3→V6 的建设过程中，同一个 rules 集合在 TRAE 的不同基座（GPT-5.2、GPT-5.4 等）上的行为一致性存在可测量的差异——这表明确实存在“未被调优的组合在实践中会受阻”的现象。这不是“版本高一点就好”的量变，而是直接决定了该组合是否满足运载前提的质变。

不过，需要澄清一个要点：非特调并不意味着完全不可用。在现有 LLM 的性能前提下，AC 范式的抽象原则即使不做平台适配，agent 在语义层面仍可能完成部分任务——语言理解能力并未消失。但“能用”不等于“可以这样用”——真正的问题是跨平台迁移时丢掉了什么。平台特有的工具和机制——例如 Claude Code 的 `claude -p` 命令、TRAE 的 AskUserQuestion 结构化框架、不同 IDE 的 subagent 类型集——在非特调环境中完全无法被利用。而优质 Skill 的本质，正是将这些平台特有机制硬化到行为中：显式写入工具调用方式、固化行为路径、消除模型自行判断的空间。一旦跨平台，这些硬化全部失效，能迁移的只剩下高度抽象的概念层——模型由此重新被迫去“悟”该怎么做。实践反复验证了：仅靠模型自行推断（而非显式程序化引导）会导致行为不可预测。

这个问题在两个层面直接暴露。

首先是触发层面。高度抽象的 Skill 描述迫使基座自己去理解“这个 Skill 是用来做什么的”。实践中的典型失败模式是——Skill 用设计者词汇（如“拓扑化拆分”“稳定替身”）描述行为，但用户在自然语言中几乎不使用这些词汇，LLM 也匹配不上这些词。实证来自 acp-skill-creator-6-1 压测中的四因子对照分析：s0402 在语义匹配中全部通过，而 s0202/s0203/s0301/s0602

全部失败，根因是描述词汇与用户的自然语言脱节（用户说”Mock/假数据”而非”稳定替身”，说”模块划分”而非”拓扑化拆分”）。当 LLM 无法识别”这个 Skill 是用来做什么的”时，它就不会触发该 Skill。

其次是执行层面。即使 Skill 被成功触发，如果其行为描述是高度抽象的原则而非可执行的步骤，基座模型会自行填补行为空白——跳过显式路由、跳过前置条件检查，由此产生锚点偏移和契约漂移。实践反复验证了一个结论：Skill 越抽象，agent 行为越不可预测。

因此，针对具体 IDE 做特调不是锦上添花的优化，而是确保 agent 行为可预测、可审计的前提条件。这里被特调的是**实现工艺**，不是 AC 范式本身所回答的问题；问题的一般性与实现的可迁移性并不是同一层级。

5.2 从 skill-creator 到 acp-skill-creator-6-1

原装 skill-creator 来自 Claude Code 生态，其设计核心假定 Claude 系列模型和 Claude Code 的工具链——Claude Code 环境无需任何配置即可使用，因为后端假设、subagent 类型假设、行为模式假设全部指向 Claude 系列模型。

但将原装 skill-creator 迁移到 TRAE 时，问题立即暴露。TRAE 的 Skill 触发机制仅依赖 description 字段做语义匹配，TRAE 只提供两种 subagent 类型，TRAE 的 AskUserQuestion 工具实现也与 Claude Code 完全不同——这些差异在 Claude Code 生态中根本不存在对应物。

原装 skill-creator 对这些平台差异的处理方式很直接——抽象到足够通用的原则层面。结果就是：除了极其抽象的原则之外几乎没办法用，基座需要大量”悟”才能执行，行为极不稳定。

acp-skill-creator-6-1 的设计哲学与此相反：它将 TRAE 的平台约束直接固化到 Skill 行为中，不依赖基座去”悟”。三个关键差异足以说明问题：

- 原装 skill-creator 依赖自然语言沟通推进关键决策——acp-6-1 强制使用 AskUserQuestion 的结构化选项框架，消除开放式问题导致的信息歧义；
- 原装 skill-creator 的 Skill 触发描述是给设计者看的——acp-6-1 的 Description 四因子模型专门针对 TRAE 的 LLM 语义匹配机制优化，尤

其是 F2 因子（描述词汇必须与用户自然语言重叠），因为这是区分语义匹配成功与失败的关键变量；

- 原装 skill-creator 的 subagent 类型集包含 TRAE 不存在的类型——acp-6-1 硬编码禁止 search subagent，只允许 TRAE 实际提供的两种类型。

这不是在原装 skill-creator 上加配置文件，而是基于 TRAE 的平台现实重新设计。这个对比直接说明了：不做针对性特调，即使概念上相同的工具也会在跨平台迁移中失效。它证明的是“平台适配是必要条件”，而不是“AC 只能存在于 TRAE 之中”。

5.3 部署工具的同源证据

deploy-ac-v6-components¹是另一个特调佐证：它硬编码了 TRAE Agent 的专属加载 URL（区分 CN 版和国际版），使用 TRAE 项目级约定的 .trae/ 路径，并内置四级降级路径以应对 TRAE 的 RunCommand 路径在特定基座或网络条件下可能失效的问题。部署工具本身遵守它要部署的规则——每步留痕到 .trae/documents/ 部署进度-note.md——这恰好是 AC 范式锚点留痕原则在 TRAE 平台上的自我使用。

5.4 非通用性声明

AC 范式 V6 当前版本是 TRAE 特调版。现有组件直接迁移到其他 agent 工具（如 Claude Code、Cursor、Windsurf）上无法确保相同的体验与有效性。这种非通用性指向的是**当前实现工艺的边界**，而不是 AC 范式作为组织原理的问题意识失效。明确定义适用边界，比模糊宣告“已经通用”更有工程价值；跨平台泛化将作为 V7 方向在第9节讨论。

¹AC 范式一键部署 Skill 开源地址：<https://github.com/jopsammy/AC-skill-deploy-ac-v6-components>

6 Subagent 与媒介强制

6.1 注意力浓缩与局部压缩

在 AC 范式的组织原理中，subagent 策略不只是工程技巧，而是一种应对媒介强制的结构手段；V6 在 TRAE 上的实现则尽可能把这一原则做到了可操作化。在单次闭环层面，ReAct 范式^[1]已论证了推理与行动交织协同的有效性；AC 的 subagent 策略将这一范式从单次闭环内的交替协同推展到跨执行断面的分布式组织。双重机制同时运作：

提高局部注意力浓度。在主线程上下文中，执行者需要同时持有 spec 约束、rules 约束、当前进度、未闭合项、多任务交叉——上下文拥挤直接导致注意力稀释。将独立检查点或关键独立任务放入 subagent，利用更干净的 context window（只携带任务指令 + 隔离约束），提高局部注意力浓度。

高质量压缩。subagent 的产出（分析报告、审查结论、方案产出）将局部执行任务做高质量压缩——subagent 在干净上下文中完成推理，浓缩的结论回传给主线程。主线程保留更多的调度空间，不必在每次需要深度分析时消耗自己的注意力预算。

这两种解释的理论基础可以回链到长程实践理论^[1]中的“媒介强制”原理：执行介质的物理上限（context window 容量、注意力持续时长）迫使任务分段。subagent 不是绕过了媒介强制，而是将媒介强制的压迫从主线程转移到了更小的封闭上下文中——利用“问题越小，上下文越干净”的信息优势。

6.2 注意力预算的分层策略

面对这一媒介强制，核心的上下文管理问题立即显现：长程任务在足够长的时序跨度下，其产生的上下文总量将超出单次窗口容量——问题不是“能不能全量保留”（不能），而是“保留什么、如何压缩、如何确保压缩后的状态仍然可用”。

subagent 的策略直接回应了这个约束：

- 主线程的 context window 只承载全局约束（两核文件判据、rules 元约束、当前 spec/plan）+ subagent 产出摘要——不承载深度推理的中间

步骤：

- 每个 subagent 的 context window 单独承载其任务所需的完整信息 + 隔离约束——推理完成后传回摘要即可释放。

这本质上是”注意力预算的分层管理”：主线程的注意力用于全局调度和约束维护，subagent 的注意力用于局部深度推理。两者不共享注意力预算池。

但不共享预算池为什么就一定更优？下面的形式化分析提供了一个可检验的解释框架。

6.3 最优性能投影与风险对冲

根据对多个长程任务的实践观察，同一个基座模型的性能表现可以通过一个特定的坐标系进行形式化描述：设横坐标为时序 t （任务步或对话轮数），纵坐标为单轮灌入上下文的增量 $c(t)$ ，坐标零点为无状态基座。这里给出的“性能投影”更适合被理解为对当前实现观察的形式化解释框架，而非已经对全部平台与全部基座完成外推验证的普适定律。

如果对这个坐标系在区间 $[0, T]$ 内进行积分（即计算曲线下方的投影面积），我们将这个面积定义为**性能投影（Performance Projection，记为 $\mathcal{P}$ ）**：

$$\mathcal{P} = \int_0^T c(t) dt \quad (1)$$

实践发现一个关键规律：对于同一个基座而言，存在一个**最优性能投影阈值 $\mathcal{P}_{\text{opt}}$** ，且该阈值基本近似恒定，与具体任务性质无关。当累积的性能投影 $\mathcal{P} \leq \mathcal{P}_{\text{opt}}$ 时，基座的推理性能在最佳区间内，表现出高度的一致性和可靠性（性能无显著区别）；一旦 $\mathcal{P} > \mathcal{P}_{\text{opt}}$ ，即突破了最优投影区间，模型的指令遵循能力、逻辑连贯性和局部注意力会发生急剧下降。

对比主线程（Main Thread）与独立子智能体（Subagent）的运行特征：

- **主线程**：作为一个长时序模型（ T_{main} 极大），虽然单位时序下的上下文增量 $c_{\text{main}}(t)$ 相对较少，但随着时序无限拉长，其性能投影 $\mathcal{P}_{\text{main}} = \int c_{\text{main}}(t)dt$ 会不断单调递增，极易突破 $\mathcal{P}_{\text{opt}}$ 的红线。

- **Subagent:** 作为短时序模型 (T_{sub} 极小), 即使为了完成独立任务在单轮中灌入较大的上下文增量 $c_{\text{sub}}(t)$, 由于积分区间极短, 其最终累积的性能投影 $\mathcal{P}_{\text{sub}}$ 依然远小于主线程, 极难触发阈值突破风险。

因此, 即使是同一个基座, 在一个足够长的时序下, **Subagent** 突破最优性能投影区间的风险远小于主线程。这意味着, 在充分提供上下文拼接的前提下, 对于局部执行性任务, **Subagent** 的性能在绝大多数情况下都是**大于等于**主线程的。充分利用 **Subagent** 策略, 从根本上说是一种对”最优性能投影突破风险”的高质量对冲策略。

值得注意的是, 即使在主线程中引入了上下文压缩、历史重置等技术使得 $c'(t) < c(t)$, 积分 $\int c'(t)dt$ 随时间增长的单调性依然不变, 上述结论仍然成立。

这一洞察充分解释了 GN-004 (独立审查机制) 高度有效的原因: 为什么同一个任务, 主线程直接做可能出错, 但用**同一个基座**拉起 **Subagent** 进行审查, 却能获得极高的审查质量? 这与我们所测试模型的表现上限无关, 而是通过物理隔离的机制, 在 **Subagent** 中重置了积分起点 ($t = 0$), 从而在局部任务中将性能投影重新拉回到了 $\mathcal{P}_{\text{opt}}$ 的安全水位之内。这证明了, 一定程度上充分的 **Subagent** 策略实际上是在更高效地压榨模型的物理性能, 而非纯粹的无效工程消耗。

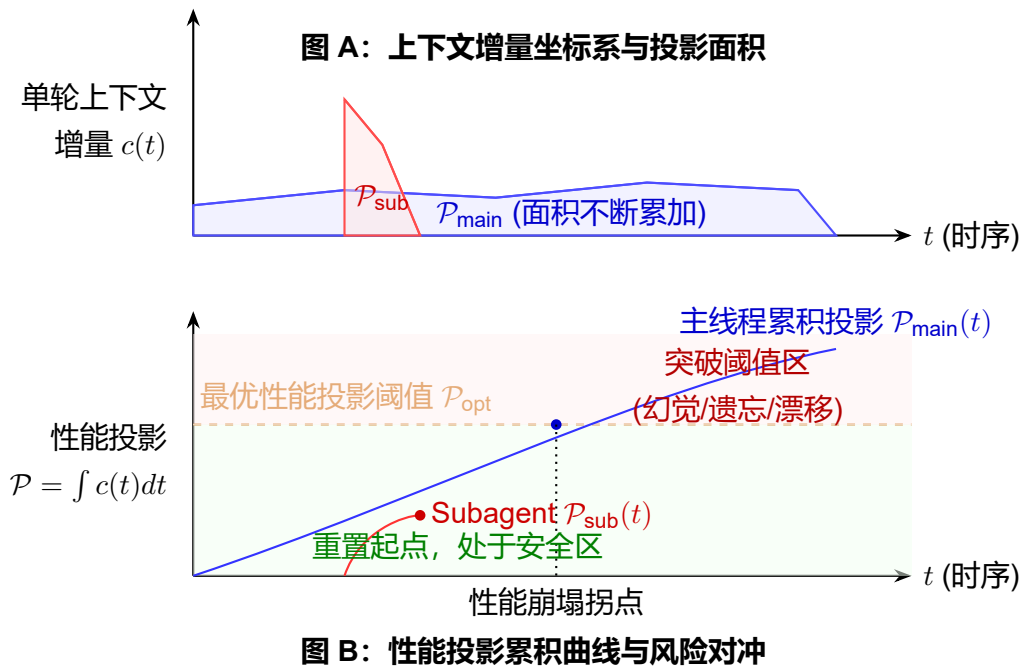


Figure 5: 最优性能投影 (P_{opt}) 与主线程/Subagent 的风险对冲曲线

6.4 并行上限的工程含义

需要说明的是，并行上限 2/3（单批并行 ≤ 2 个 subagent，全局并行 ≤ 3 个含非并行分支）是 AC 范式 V6 在 TRAE 参考系下形成的实现参数，而不是 AC 范式定义中的普遍常数。这个数字基于以下三个约束：

- **平台约束：**TRAE 禁止 subsubagent 嵌套调用。这意味着如果一个 subagent 在运行中需要拉起另一个 subagent 进行独立验证（如并行盲点分析需要 GN-004 审查），它无法直接做到——只能返回主线程后由主线程调度。并行数越大，这种“返回 → 调度 → 再拉起”的链路延迟累积越严重。
- **注意力预算：**每个并行 subagent 消耗主线程的调度注意力。当主线程同时管理 4 个以上的并行 subagent 时，它自身的上下文开始拥挤——调度质量下降，反而抵消了并行带来的效率增益。

- **失效隔离**：并行数越大，单个 subagent 失效时需要回滚的波及面越大。2/3 上限确保在任何时刻最多只有 2~3 个 branch 处于活跃状态，失效隔离的操作复杂度可控。

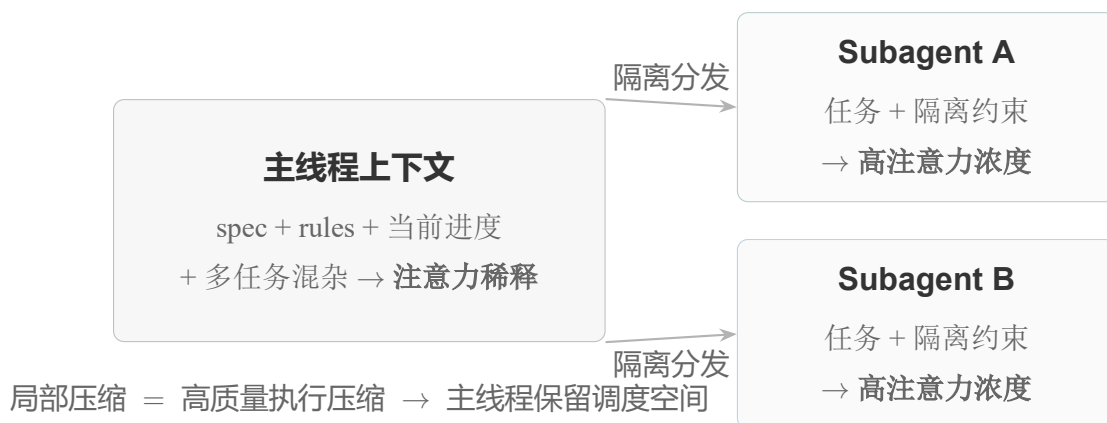


Figure 6: Subagent 注意力浓缩示意图

7 基建噪声与平台约束

7.1 基建噪声的结构性影响

本节讨论的不是 AC 范式本体的定义要件，而是 AC 在具体平台落地时所遭遇的环境噪声与实现约束。在长程实践过程中，基建噪声会对任务性能造成巨大影响。这里的基建噪声包括四类：

1. **本地机性能**：CPU 和 GPU 性能直接影响 agent 的实际表现。在长程任务中，慢 30% 的单次响应时间经过数百次工具调用的累积，会显著改变执行者的注意力窗口和人类的等待耐受度。
2. **agent 与终端链路的通畅性**：TRAE 的 agent 通过终端（terminal）执行命令、读取输出。终端链路的不稳定（命令执行超时、输出截断、编码问题）在长程任务中不是偶发事件，而是反复出现的干扰源。每次终端链路故障都消耗一次上下文恢复——故障频率高到一定程度时，执行节奏被持续打断。

3. **基座与工具链的稳定性：**不同基座（GPT-5.2/GPT-5.4/DeepSeek-V4 等）对同一工具调用协议的遵循程度不同。在 V5.3→V6 建设期间观察到：同一个 Skill 的 Action Flow 在某个基座上完整执行，在另一个基座上跳过了其中两个步骤——且跳过行为不可复现。这不一定是基座的”bug”，而是工具调用链在长程压力下的退化性行为。
4. **基座 API 的连接稳定性：**API 超时、限流、服务降级等不可控因素在单次调用中影响有限，但在数小时至数天的长程任务中被数百次调用持续放大。AC 范式 V6 的锚点契约机制（current-note、spec 三件套）虽然在机制层面上保证了 API 中断后的状态续接，但频繁的中断-恢复循环仍会持续侵蚀 agent 的整体性能——每次恢复都需要额外消耗上下文窗口重新加载状态文件，每次中断都打断执行者的注意力连续性。这种”可续接但持续磨损”的效应，是长程工程中特有的隐性成本。

前三类基建噪声在理论上的统一解释是：长程实践的运载前提要求 LLM 引擎满足最低指令遵循（E3）和注意力预算非单调衰减（H1）两项硬约束^[1]。基建噪声的累积效应使得这两项约束在真实环境中不是”满足或不满足”的二值判定，而是”在什么噪声水平下仍能满足”的连续退化。

7.2 102 小时案例中的基建噪声实证

第 8 节的 102 小时案例提供了基建噪声影响长程实践的精确实证。判题机作为外部执行环境，引入了四类本地无法复现的噪声：

- **判题机单核：**v7 OpenMP 多核并行（本地 4 核 3.56s）在判题机完全无效——判题机以单核执行，多线程优化全部白费。
- **判题机不支持 AVX：**v8 AVX2 编译失败（无-mavx2 支持），v8b 使用 `__attribute__((target("avx2")))` 隔离，结果判题机旧编译器产生错误代码——0/19 全错。
- **判题机微架构效率低约 15%：**同样的 418 亿次操作，本地 Zen4 8.89s vs 判题机 >10s。这 15% 的差异决定了 v11（已达纯标量极限）是否能通过——最终答案是不能。

- **判题机 target 属性不兼容**：编译器对特定 target 属性的支持差异导致本地通过但判题机全错——这是典型的”不同执行环境间的隐式契约不一致”。

这四类噪声的共同特征是：本地无法复现，只有提交判题后才能知道结果。每次判题轮次消耗 ~15 分钟（代码修改 + 编译 + 提交 + 等待判题）——5 轮判题失败合计消耗了 ~75 分钟的工程时间。在 102 小时的总时间跨度中，这 75 分钟看似不多，但它的影响不在于时间本身，而在于”连续 4 次提交失败”对执行者判断力的侵蚀——v7→v8→v8b 三轮全错后，执行者已经倾向于”放弃硬件优化路线”（事实上 v11 最终证明纯标量优化在正确方向上——只是判题机环境不允许）。

7.3 基建噪声不是偶发问题

基建噪声不会随版本迭代自然消失。在长程实践的框架下，执行环境的参数空间在第 i 步执行前在工程上难以穷举——而外部漂移的定义是”外部世界独立变化，且变化在步骤 i 执行前不可知”。

只要外部执行环境（判题机、CI/CD pipeline、第三方 API）的完整参数空间在任务执行前不可穷举，基建噪声就始终是长程实践的结构因素——不是”做得更多就会消失”，而是”做每个任务前都需要为它留出冗余”。

AC 范式 V6 对基建噪声的工程回应分两个层面。需要强调的是，AC 的作用不是消除噪声本身，而是在噪声不可避免时，仍让状态续接、降级路径与错误隔离保持可组织：

- **设计层面**：多级降级路径（deploy 的四级降级、GN-004 的不可用降级）——不是假设一切正常，而是预置”某级失效后如何退一级继续”的路径。
- **机制层面**：current-note 中显式记录基建噪声导致的路线变更和”已证伪”结论——使执行者不会在跨天恢复后重新走已被基建噪声证伪的死路。

基座 API 的连接稳定性在这一框架下是一个需要单独审视的子类：锚点契约机制保证了 API 中断后状态可续接——任务不会因一次中断而丢失

全部进度。但它无法消除中断-恢复循环对注意力预算的持续消耗：每一次恢复都意味着额外加载状态文件、重建上下文锚点，在十小时以上的任务中，这些”可恢复但不可忽略的磨损”会实质性降低有效工作时间占比。这恰恰说明 API 稳定性必须被列为独立的基建噪声源，而不能被归类为”已被锚点契约机制解决的偶发问题”。

第 9 节将把”本地机性能”和”平台约束”作为 V7 方向的起点，讨论性能隔离和跨平台泛化的必要性。

8 102 小时案例：AC 范式下的长程实践

AC 范式 V6 目前已在真实的商业工程（如跨周期的系统重构、多模块并行重写等）中进入常态化运转。然而，真实的业务代码库受限于严格的商业保密义务无法公开，且业务逻辑的发散性往往会掩盖协同机制本身的骨架。相比于业界许多仅停留在演示（Demo）阶段、一旦接入真实高噪工程便迅速退化的长程 Agent 方案，AC 范式在本文讨论的实现环境中是作为经受日常高频实战考验的底层基建存在的。

为了在合规的前提下提供一个逻辑闭环、变量受控且完全可复现的证据标本，本节特意剥离了业务领域的繁杂细节，选取了一道在 AC 范式 V6 的 TRAE 实现中由 DeepSeek-v4-pro 持续 102 小时攻克的组合数学问题为例。²这并非偶然的孤例，而是 AC 范式日常运转效能的一次极压切片。本节不仅解释”为什么这道题构成了严苛的长程实践”，更解释”为什么 AC 范式的锚点—契约—审查结构能在当前实现环境中支撑这次跨 6 天的持续解题并最终成功”。需要说明的是，这一案例的证据功能在于展示结构机制如何工作，而不在于单独构成跨平台普适性的统计证明。

8.1 案例概览

题目是一道 $N = 5000$ 的组合数学/数论变换（NTT）算法题，时间限制 10s，模数 998244353（NTT-friendly）。

²原始研究仓（含完整解题记录、锚点文档、判题日志与 35+ 产出物）：https://github.com/jopsammy/task1_raw_repo

问题定义。称长度为 $2n$ 、仅由 (与) 构成的括号串 s 为合法，当且仅当任意前缀左括号数不少于右括号数，且整串左右括号各有 n 个。对合法括号串 s ，该题定义了两组加权函数：

- $\text{alice}(s)$ 为 s 中等于 () 的子序列个数——即满足 $1 \leq i < j \leq 2n$ 、 $s_i = ($ 、 $s_j =)$ 的二元组 (i, j) 的数量； $\text{SA}(s) = a^{\text{alice}(s)}$ 。
- $\text{bob}(s)$ 通过一个逐前缀批次扫描的迭代过程定义， $\text{SB}(s) = b^{\text{bob}(s)}$ 。

对每个 i ($1 \leq i \leq n$)，设 $\mathcal{D}_i$ 为所有长度 $2i$ 的合法括号串集合，目标为计算：

$$F_i = \sum_{s \in \mathcal{D}_i} \text{SA}(s) \times \text{SB}(s) = \sum_{s \in \mathcal{D}_i} a^{\text{alice}(s)} \times b^{\text{bob}(s)} \pmod{998244353}.$$

输入为一行三个整数 n, a, b ($n \leq 5000, 1 \leq a, b < 998244353$)，输出为一行 n 个整数 $F_1, \dots, F_n$ 。闭合判据完全可机械判定：21 个测试用例全部通过、运行时间 $\leq 10\text{s}$ 、内存 $\leq 1\text{GB}$ 、单文件实现、不使用非标准降级策略——不存在”方向对了”等模糊中间态。

这道题的结构要害在于：Alice 的计数（子序列配对）与 Bob 的计数（前缀批次扫描）是两种性质完全不同的组合统计量。将它们统一到同一个递推框架，需要一种能同时兼容树结构（Alice 沿 Catalan 分解）和 x 序列结构（Bob 沿前缀扫描）的分解范式——而这个范式在任务启动时无法预知，只能在穷举多条候选路线、逐条验证之后才能被固定。

DeepSeek-v4-pro 在 AC 范式下从 2026 年 5 月 20 日持续解题至 5 月 25 日，共完成 16+ 轮判题迭代，从 $O(N^3)$ Python OOM 开始，最终以 v15-b 二维自适应 NTT 实现 21/21 全通。

这 6 天不是”一道很难的算法题碰巧做了很久”——它的解题路径中存在典型的不可预计算顺序依赖链。两个关键突破点分别发生在第 76 小时和第 86 小时（而非第 1 小时），因为它们依赖的是不执行到那一步就无法预先获得的认知积累。13 条失败的数学路线、5 种完全不同的分解范式，最终突破不是来自更强的单次推理，而是来自跨检查点的状态积累和盲点发现。以下各小节依次回答设计自证所需的关键问题。

8.2 为什么它是长程实践

先做框架判断，再展开时间线。

8.2.1 不可预计算顺序依赖链：wrapping 的结构性屏障

在 13 条失败路线中（路线 A~K + L-DP 降阶 A/B/C），5 种完全不同的分解范式——树分解、x-seq 分解、c-based 递推、L(i) 序列 DP、生成函数——均在 wrapping 步骤遭到了共同的结构性障碍。wrapping 操作 $A \rightarrow (A)$ 需要完整的高度函数：对当前 Dyck 词计算其全部高度取值。步骤 $N + 1$ 的关键输入（高度函数在给定词上的完整取值）只能在步骤 N 执行后才可获得，且该输入的维度为 $O(2^k)$ ，不可低维压缩。任何试图用 $O(1)$ 或 $O(k^c)$ 维度的 DP 状态预计算 wrapping 结果的尝试都因信息维度不匹配而失败。

这是典型的长程实践中“不可预计算顺序依赖链”的精确实例：步骤 $N + 1$ 的输入不可在步骤 N 执行前被静态折叠。wrapping 不是计算量问题——增大上下文窗口无法折叠尚未生成的信息。13 条路线、5 种范式的共同死因不是智力不足，而是结构性的信息缺口：wrapping 的全局性意味着局部信息在结构上不足以闭合下一步——增大计算量无法填补这个信息缺口。

8.2.2 两个关键突破都发生在“不执行就不会知道”的位置

(q,t)-Catalan 等价发现（第 76 小时）：通过将题目定义的括号串权重与 Dyck 路径的标准几何量逐项对照，于第 76 小时发现题目等价于 (q,t)-Catalan 数的特定实例^[12-13]。这个等价关系不是“一开始就写在题目里”的——它依赖前 75 小时的路线尝试中积累的 wrapping 结构认知。没有 13 条路线的失败经验，在我们观察的案例中将无法识别出这个等价。

NTT 盲点发现（第 86 小时）：在将纯标量优化版本 $v1 \rightarrow v11$ 压到极限（ $O(N^3/6)$ 从 77s 降至 8.89s）后，定量计算出在判题机单核 10s 的约束下，纯标量路线无法完成全部 208 亿次点积。这个“物理不可行”的定量证据触发了关键盲点意识——执行者意识到必须放弃纯标量路线，转向代数降维。随后的盲点分析发现了 q-阶乘分离 +NTT 卷积路径，将 $O(N^3)$ 瓶颈化为 NTT 卷积。

两个突破的时间位置：(q,t)-Catalan 等价依赖前 75 小时的失败经验积累，NTT 盲点依赖第 60-86 小时的工程极限压榨。两者都是“不执行到那一步就不知道”的认知缺口——在任务开始时，无法预判第几小时会发现什么。

8.3 案例时间线

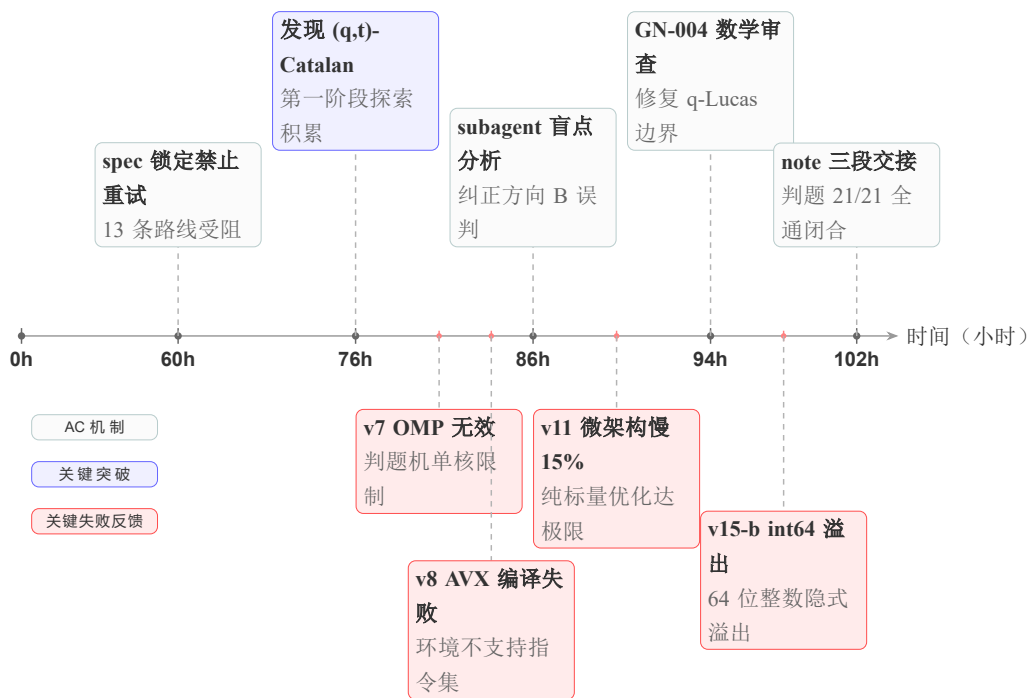


Figure 7: 102 小时案例推进链图

图7以时间轴展示了 102 小时中的关键节点。以下按时间顺序补充各阶段的结构性背景。

8.3.1 第一阶段 (0–60h): 穷举路线

执行者在 spec.md 锚定下，尝试了 13 条数学路线。每条路线的闭合标准在 tasks.md 中预定义，每条路线的失败根因在 current-note.md 中记录。13 条路线共同指向 wrapping 的结构性不可行性——这一结论在第 60 小时被写入 spec.md 的”已证伪事实”并标注为”禁止重试路线”。

8.3.2 第二阶段 (60–86h): 工程极限压榨

在明确数学路线受阻后，执行者转而尝试工程优化路线:v1(朴素 $O(N^3/6)$ Python OOM)→v2(C++ 移植, 77s)→v3/v4(三角矩阵 + 对角线优化)→v5/v6

(4 路/8 路 ILP 展开) → v7 (OpenMP 多核并行——但判题机单核, 无效) → v8/v8b (AVX2——但判题机不支持, 0/19 全错) → v9/v10 (循环交换 + 寄存器优化) → v11 (32 路 ILP, 8.89s, 已达纯标量极限)。

v11 的 8.89s 在本地通过, 但判题机 (微架构效率低约 15%) 上超出 10s 限制。这个跨平台差异直接触发了第 86 小时的盲点突破: 执行者定量确认了纯标量路线在物理上不可行。

8.3.3 第三阶段 (86–102h): 代数降维与最终收敛

第 86 小时盲点分析发现 q-阶乘分离 + NTT 卷积路径 (v14), 将 $O(N^3)$ 瓶颈化为 NTT 卷积。v14 首次判题 7/21 正确、14/21 WA——根因是 q-阶乘零因子 bug (a 有小乘法阶时 $\text{Fact}[i]=0$ 导致 0/0 除法)。修复为双路径: NTT + v11 回退。

第 94 小时 GN-004 对 v15 的 q-Lucas 分解公式进行两轮独立数学审查: 第一轮发现索引偏移, 第二轮发现借位遗漏——两次错误均在 C++ 实现前修正。v15 判题 19/21、2/21 WA——signed int64 溢出。v15-b 修复为 unsigned __int128, 290/290 交叉验证全通, 21/21 判题全通。

8.4 为什么 AC 结构在该实现中能支撑它

在我们的实验范围内, 未观察到任何基座模型在裸连 API 条件下成功解出该问题——问题本身的不可预计算顺序依赖链意味着, 任何基座在脱离结构约束后, 都会在 10 轮以内因跨断面上下文断裂而不可逆转地偏离正确方向。而一旦在本文讨论的 TRAE 实现中加载 AC 范式的锚点—契约—审查三层结构, DeepSeek-v4-pro 即在 102 小时内完成 16+ 轮判题迭代并最终闭合。这一结果支持了“结构约束具有必要性”的判断, 但并不单独证明 AC 是唯一可行路径; 它更准确地说明, AC 范式所识别的那组三层结构在真实 agent 平台上确实能够形成有效支撑。

更具体地说, 此处需要区分两类因素: 其一是 AC 范式提供的组织结构, 如锚点、禁止重试路线、独立审查与隔离调度; 其二是当前实现环境所提供的执行载体与噪声背景, 如 TRAE 的工具链、DeepSeek-v4-pro 基座、判题机约束与平台交互边界。案例成立依赖二者共同出现, 但其中真正构成本文论证核心的, 是前者如何在后者的现实噪声中维持长程连续性。

锚点文档维持跨断面状态连续性: 四份锚点文档——spec.md (已证真/已证伪事实 + 禁止重试路线 + GATE 表)、tasks.md (依赖 DAG + 闭合判据)、checklist.md (三层验证)、current-note.md (1550+ 行研究日志 + 作废索引)——使执行者 (和人类观察者) 在 6 天跨度中每次恢复上下文时不需要重读全部代码。作废索引 (9 条被推翻结论 + 推翻原因 + 推翻日期) 防止了新执行者引用已被证伪的中间结论。

禁止重试路线防止资源浪费: spec.md § 三显式列出 9 条禁止重试路线, 每条附带失败根因。这使得执行者在跨天后恢复时不会重新尝试已被证伪的路径——”负面知识的结构化沉淀”是 AC 范式区别于普通 prompt engineering 的关键差异。

GN-004 独立审查阻止了数学错误进入实现: v15 的 q-Lucas 分解公式经过 GN-004 两轮独立数学审查——第一轮发现索引偏移 ($\binom{r_0+k_0}{r_0}_a$ 应为 $\binom{r_0+k_0-1}{r_0}_a$), 第二轮发现借位遗漏 ($k_0 = 0, r_0 = 0$ 时产生借位 $\binom{r_1+k_1-1}{r_1}$, 不可遗漏)。两次发现的错误都是推导中容易遗漏的边界条件, 如果直接进入 C++ 代码将至少浪费 2 轮判题 (~30 分钟工程时间)。

Subagent 隔离的盲点分析发现了主执行者遗漏的突破方向: 0524 盲点分析纠正了”方向 B (NTT) 是死胡同”的误判——关键洞察是同一技术 (NTT) 在不同位置 (L(i) DP vs Garsia-Haglund 内层) 的适用性不同, 但这种区分在主执行者上下文中被”方向 B 已死”的标签压制。0525 盲点分析证伪了”q-Lucas 必然常数爆炸”的结论——发现余数层求和也是互相关可 NTT 降维。

两次盲点分析的价值在独立性: 不受主执行者已有认知框架的约束, 以干净上下文重新审查问题。

8.5 哪些机制在起作用

Table 2: AC 机制在 102 小时案例中的使用证据

AC 机制	案例中的具体使用	证据定位	
spec.md	已证真 22 条事实（所有执行者必须遵守）、已证伪 13 条路线（含共同死因）、禁止重试 9 条（防止走回头路）、GATE 对照表、判题反馈 17 轮汇总	spec.md § 一 ~ § 六	
tasks.md	子任务依赖 DAG、每任务闭合判据、版本演进记录（v1→v15-b）、已关闭路线表	tasks.md S0-1~S0-7	
checklist.md	全局出口闸门（GATE-1~5）、数学正确性 67 项 checklist、性能基准（v11/v14/v15 对照）、合规化 C1-C6	checklist.md 全部章节	
current-note.md	1550+ 行研究日志：作废索引（9 条）、GATE 对照表、判题反馈历史、已证真/已证伪事实、13 条路线完整记录、版本演进、产出物清单（35+ 文件）	current-note.md § 一 ~ § 十九	
GN-004	两轮数学审查（q-Lucas 索引偏移 + 借位遗漏），均在 C++ 实现前完成	current-note.md	L1476-L1491
Subagent（盲点分析）	0524 分析纠正”方向 B 死胡同”误判 →v14 突破；0525 分析证伪”q-Lucas 常数爆炸”→v15 突破	blind-spot-analysis-note0524/0525.md	
三段交接结构	current-note.md § 十七显式标注工程过程/交接状态/最终结果三段，闭合注记含操作计数实测 + 物理可行性分析	current-note.md	L1294-L1355

表2列出了在案例中实际发挥作用的 AC 机制及其使用证据。盲点分析在本次实践中以独立 `note` 文件形式完成，未使用 `formal subagent` 调度框架——但这不影响盲点分析作为独立审查机制所发挥的关键作用：两次盲点分析均在干净上下文中完成，不受主执行者已有认知框架约束，分别发现了主执行者遗漏的 NTT 突破方向和 q-Lucas 降维路径。

9 局限性与 V7 方向

AC 范式 V6 不是终点。本节列出的不是 AC 范式作为抽象组织原理的全部“理论局限”，而是其在 TRAE 参考系下这一当前实现所暴露的 7 项工程边界。其中，第 1 项（TRAE 特化）属于 AC 自身尚未完成的工程任务，可以通过平台适配层在 V7 中推进；其余 6 项——`subsubagent` 嵌套、`agent id` 追溯、系统元信息可观测、状态快照回滚、性能隔离、工具调用配额——暴露了当前 TRAE 平台在这些维度上的能力边界。这些局限的方向是明确的，但其突破路径涉及平台层面的能力演进，超过了 AC 作为上层组织范式的工程范围。换言之，V7 不是对 V6 的自我否定，而是从“单平台实现”走向“实现族扩展”的下一步。

9.1 局限 → V7 方向映射



Figure 8: TRAE 约束 → V7 方向映射图

图8列出了 7 项局限与对应方向。其中第 1 项（TRAE 特化）对应的是 AC 自身尚未完成的跨平台适配问题；虚线标注的后 6 项暴露了当前平台在 subagent 调度、元信息可观测、状态管理、性能隔离和资源配额等维度上的能力边界——方向明确，突破需要平台侧的能力供给。它们限制的是 AC 在 TRAE 上的当前实现能力，而不构成对 AC 所提出之组织问题本身的反证。以下逐项展开每项局限的现状、根因，以及对应的解决方向。

9.1.1 TRAE 特化 → 跨平台泛化

现状：AC 范式 V6 的所有组件——两核文件、7 Rules、11 Skill、GN-004、deploy-ac-v6-components、acp-skill-creator-6-1——全部围绕 TRAE 的 agent 工具特化定制。硬编码路径约定（.trae/）、TRAE 专属 Agent URL、description 四因子模型、并行上限（2/3）、四级降级路径中的 TRAE CLI 调用——这些设计使得 V6 在 TRAE 上”安装即用”，但在其他 agent 平台（如 Cursor、Cline、Copilot Chat 等）上无法直接迁移。

TRAE 特化是 V6 的设计选择：第 5 节已论证，基于 AC 范式思想和长程实践理论，必须围绕”agent 工具 + 基座组合”做独立测试与调优。未被调

优的组合在实践上会受阻、在理论上也能找到对应原因。

V7 方向：跨平台泛化。将 V6 中与 TRAE 耦合的部分抽象为平台适配层（platform adapter），使 AC 范式的核心组织逻辑（能力锁、Rules 层次结构、GN-004 审查协议、锚点体系）与具体平台的工具名、路径约定、Agent 调度机制解耦。泛化不是“去掉 TRAE 特调”——而是“对每个平台做同样深度的特调，但共享同一套组织逻辑”。

9.1.2 禁止 subsubagent→subsubagent 嵌套支持

现状：TRAE 不允许在 subagent 内继续拉起 subsubagent。这是平台用来防止失控嵌套导致 token 爆炸的保护机制。但从 AC 范式的角度看，这个约束直接阻碍了“极致利用 subagent”的组织上限。

具体影响在两个场景中最为明显：(1) GN-004 审查链——当前审查 → 修正 → 复审的循环中，修正步骤在主线程执行而非在独立审查的隔离上下文中完成，降低了“无知之幕”的持续强度。(2) 盲点分析链——subagent 发现的问题如果需要进一步分解为子分析，当前只能将中间结果传回主线程再由主线程分配新 subagent，增加了上下文污染和调度开销。

解决方向：需要平台侧提供有限深度的 subsubagent 嵌套调度能力。嵌套隔离属于平台调度机制层面的供给——设置嵌套深度上限（如 2 层），嵌套 subagent 继承父级的 core rules 和 tasks 约束但不继承当前执行状态。嵌套机制的开放程度和 token 消耗控制粒度，属于平台侧的设计范畴。

9.1.3 agent id 能力不均 →agent id 统一可追溯

现状：在 TRAE 环境中，只有内置 GPT 模型（5.2、5.4）稳定可见 agent id。其他基座（如 DeepSeek-v4-pro）无法稳定获取 agent id。根因归属当前不可判定——可能是 TRAE 的平台约束（仅对特定基座暴露系统元信息），也可能是 GPT 模型独有的能力（模型侧可感知并回传该元信息）。

102 小时案例中，盲点分析以独立 note 文件形式存在，未通过 formal subagent 调度框架，agent id 无从追溯——使得“该分析在独立 subagent 上下文中完成”这一判断无法被独立验证。subagent 调度台账（rules-0 § 四-11）的完整性也因 agent id 能力不均而受到制约。

解决方向：平台侧统一 agent id 的可观测输出。agent id 由平台调度层产生，属于平台侧的信息供给范畴——无论使用何种基座，将其以稳定的可观测信号对齐输出，使其成为 subagent 调度的标准元信息字段。当平台统一输出该信号后，AC 即可将其纳入 subagent 调度台账的标准字段。

9.1.4 系统元信息不可见 → 系统元信息可观测

现状：除了 GPT 模型外，主线程无法稳定观察到系统信息，包括：Skill 是否已载入、当前已加载的 Rules 列表、可用的工具类型清单、当前 subagent 的调度状态。这一问题与 agent id 能力不均联动：系统元信息不可见意味着主线程对系统状态机的可观测性不足，影响了调度精度和异常恢复能力。

在第 6 节讨论的 subagent 调度策略（注意力浓缩 + 局部压缩）中，调度决策的前提取决于主线程对“当前哪些 subagent 在运行、哪些 Skill 已载入”的实时感知。当这个前提不可靠时，调度决策只能依赖“预估”而非“观测”——这在长程任务中积累为系统性偏差。

解决方向：平台侧开放系统元信息查询接口。AC 需要一套平台无关的查询接口来获得：当前活跃 Rules 清单、已载入 Skill 清单、可用工具类型与当前工具调用配额、活跃 subagent 列表及其状态。当前这些信息由平台内部管理。当平台开放相应的查询接口后，AC 即可在其基础上构建更精准的调度逻辑。

9.1.5 回滚/续接能力缺失 → 状态机快照 + 灵活回滚

现状：TRAE 当前不支持子任务级别的状态机快照和灵活回滚。一旦 subagent 执行中出现基建噪声（终端超时、输出截断、工具调用失败），该 subagent 的上下文状态不可恢复，只能重新启动。也无法针对某个已完成的检查点做局部回滚——在当前平台下，回退操作需要重新执行整个任务段。

在 102 小时案例中，判题机的 5 轮失败导致大量工程时间被消耗在“重新提交判题”上，而非“算法设计”上。如果存在子任务级状态快照，执行者可以在 v11（纯标量极限）处创建快照，在 v14（NTT）方案失败时快速回退到 v11 的工程基线，而不需要从 spec.md 重新对齐全部上下文。

AC 范式本身的锚点体系（spec/note/.trae/documents 的一式三份交接结构）已经在设计层面对这个问题做了补偿：它们在“跨断面”层面提供了状

态恢复能力（隔天重新打开 project 时能接续）。但它们不能补偿”同一断面内”的实时回滚——当前锚点体系的恢复粒度是”任务检查点”级别，而非”subagent 内部执行状态”级别。

解决方向：平台侧提供子任务级状态快照与回滚能力。AC 的锚点体系已经在”跨断面”层面补偿了一部分状态恢复需求。平台级的实时回滚涉及存储、上下文序列化和调度状态恢复，属于平台架构层面的命题——在分布式系统的传统中，状态快照与全局一致状态恢复已有成熟的理论基础^[4]。将这一框架迁移到 agent 执行上下文中，在任务检查点创建可恢复的状态快照（含锚点状态、Task 闭合清单、subagent 执行记录），允许执行者在后续步骤失败时回滚到最近的快照点。当平台提供快照能力后，AC 的锚点体系可与其形成互补。

9.1.6 本地机性能 → 民用机性能优化

现状：本地机 CPU 与 GPU 性能直接影响 agent 整体实际表现。这不是”环境噪声可忽略”的边角因素——第 7 节已从理论和实证两个层面论证了基建噪声的结构化影响。在 102 小时案例中，本地 Zen4 与判题机之间的 ~15% 性能差异直接决定了一条优化路线是否可行。

更关键的问题在于：当前 TRAE 平台在民用机上的性能表现远未达到硬件上限。实测数据暴露了这一问题³：在 3 个并行 SubAgent 运行（无本地推理，API 链接远端模型）时，CPU 均值仅 ~30% 但峰值可达 99%，GPU 占用 327MB 但利用率仅 4%。CPU 均值的持续占用由 trae-helper 与 ai-agent 守护进程主导（15–25%），CPU 峰值由 Chat Panel DOM 无虚拟化导致的 Recalculate Style 瓶颈主导（30–60% 峰值贡献）；GPU 内存占用则源于 UMA 架构下 Electron 渲染管道的纹理累积。更反直觉的是，”退出微信后 Trae 加速”这一现象被归因为 V8 GC 频率降低与 Windows Working Set 裁剪停止两个机制——说明平台自身的运作机制（而非硬件算力）才是当前民用机性能瓶颈的主导因素。

换句话说，不是民用机性能不够，而是平台尚未充分压榨民用机已有的 CPU、GPU 和内存资源。

³见《Trae IDE 并行 SubAgent 场景下的 CPU/GPU 资源消耗分析与微信退出加速效应》（2026 年 5 月），Sammy Zeng。

解决方向：平台侧从运作机制层面做民用机性能优化。具体包括：Chat Panel 引入 DOM 虚拟化以消除 Recalculate Style 瓶颈，trae-helper 和 ai-agent 守护进程的 CPU 占用率优化，Electron 渲染管道的 GPU 内存管理优化，以及跨进程资源竞争（如微信与 Trae 之间的 Working Set 争夺）的主动规避策略。当平台将民用机的 CPU、GPU 和内存性能充分压榨后，AC 范式的调度策略和降级路径可以在更稳定的执行基线上运作，而不需要依赖云端迁移或硬件升级来解决性能问题。

9.1.7 工具调用配额硬上限 → 工具调用配额弹性扩展

现状：TRAE 对单次对话的工具调用次数存在硬上限——默认 200 次，手动调整后最大仅 500 次。在 AC 范式的长程实践语境下，即使是单次简单长程任务，工具调用量超过千次也是常态；102 小时案例的调用量则远不止于此。配额耗尽后，agent 性能显著下降——注意力被迫从任务本体转移到“如何用更少工具完成剩余步骤”的生存策略上，执行质量随配额逼近上限而递减。更严重的是，基座模型在配额用尽后被迫使用 CLI 命令或脚本强行绕过 TRAE 的工具调用通道——这不仅降低了 agent 的组织可控性，也使 AC 范式依赖的系统化工具调度（subagent 启动、GN-004 审查、Skill 调用等）在配额边界处降级为难以追溯的原始命令行操作。

配额边界还使得“注意力浓缩 + 局部压缩”的调度策略无法覆盖完整任务周期——调度器在接近配额上限时必须提前收束决策，无法按最优节奏调度后续 subagent。这与第 6 节讨论的调度精度问题形成联动：调度精度已经受限于系统元信息不可见，配额硬上限进一步压缩了调度器的可用决策空间。

解决方向：平台侧提供工具调用配额弹性扩展。将单次对话的工具调用上限提升到适配长程任务的水平，或提供配额自动续期机制。配额扩展涉及平台的资源分配策略与成本控制，属于平台侧的设计范畴。AC 可以明确长程任务对工具调用量的需求边界，在平台提供弹性配额后在其上优化调度策略。

9.2 归因与边界

表3汇总了 7 项局限的归因与方向状态。除第 1 项属于 AC 自身工程范围外，其余 6 项暴露了平台在相应维度上的能力边界。

Table 3: 局限归因与方向状态汇总

方向	归因	状态说明
跨平台泛化	AC 工程范围	平台适配层架构已可讨论，具体平台待逐一验证
subsubagent 嵌套	平台约束	嵌套深度与 token 管理策略属于平台调度机制范畴
agent id 统一可追溯	平台约束	agent id 由平台调度层产生，属于平台信息供给范畴
系统元信息可观测	平台约束	元信息由平台内部管理，属于平台可观测性范畴
状态机快照 + 灵活回滚	平台约束	涉及平台存储与调度状态恢复，属于平台架构范畴
民用机性能优化	平台约束	agent 运作机制需针对民用机 CPU/GPU/内存做深度优化
工具调用配额弹性扩展	平台约束	配额策略与资源管理属于平台资源分配范畴

这 7 项方向中，第 1 项是 V7 可以实际推进的工程任务；后 6 项的突破需要平台侧的能力供给。AC 的当前策略是：在既有约束下通过锚点体系 (§4)、调度策略 (§6) 和降级路径 (§7) 最大化组织效能，同时为每一项平台约束记录清晰的需求信号——当相应能力在平台侧可用时，AC 可以基于既有的需求信号和设计积累快速适配。

10 致谢

- 感谢杭州市上城区钱江新城中央商务产业社区在调研对接、基础压力测试样例与真实业务用例方面提供的关键性支持

- 感谢深度求索（DeepSeek）提供 Deepseek-v4-pro 基座支持；102 小时压力运行由该基座在锚点契约式人机协同工程范式（AC 范式 V6）下完成
- 感谢杭州市协同引荐平台社区（SRO）提供实质性用例支持
- 感谢彗星计划（Comet Project）成员提供技术支持
- 感谢 AI-Zion 社区大量成员提供了富有启发性的讨论，给出宝贵的建议
- 感谢 TRAE 提供关键 IDE 环境技术支持
- 感谢 Cherry Studio 提供关键技术支持

References

- [1] ZENG Q, WEI D. 长程实践：超越单次能力的结构下限[Z/OL]. 2025. <https://doi.org/10.5281/zenodo.20470866>.
- [2] LICKLIDER J C R. Man-Computer Symbiosis[J/OL]. IRE Transactions on Human Factors in Electronics, 1960, HFE-1(1): 4-11. DOI: [10.1109/THFE.2.1960.4503259](https://doi.org/10.1109/THFE.2.1960.4503259).
- [3] ENGELBART D C. Augmenting Human Intellect: A Conceptual Framework[M]. Stanford Research Institute, 1962.
- [4] CLARK A, CHALMERS D J. The Extended Mind[J/OL]. Analysis, 1998, 58(1): 7-19. DOI: [10.1093/analys/58.1.7](https://doi.org/10.1093/analys/58.1.7).
- [5] HUTCHINS E. Cognition in the Wild[M]. MIT Press, 1995.
- [6] SIMON H A. A Behavioral Model of Rational Choice[J/OL]. Quarterly Journal of Economics, 1955, 69(1): 99-118. DOI: [10.2307/1884852](https://doi.org/10.2307/1884852).
- [7] LIU F, LV B, LIU Y. From Observer to Agent: On the Unification of Physics and Intelligence Science[Z/OL]. 2024. <https://www.preprints.org/manuscript/202410.0479/v1>.

- [8] RAWLS J. A Theory of Justice[M]. Revised. Harvard University Press, 1999.
- [9] HUTCHINS E. Material Anchors for Conceptual Blends[J/OL]. Journal of Pragmatics, 2005, 37(10): 1555-1577. DOI: [10.1016/j.pragma.2004.06.008](https://doi.org/10.1016/j.pragma.2004.06.008).
- [10] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering[C]//Advances in Neural Information Processing Systems: vol. 37. 2024.
- [11] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]//International Conference on Learning Representations (ICLR). 2023.
- [12] GARSIA A M, HAIMAN M. A Remarkable q, t -Catalan Sequence and q -Lagrange Inversion[J/OL]. Journal of Algebraic Combinatorics, 1996, 5(3): 191-244. DOI: [10.1023/A:1022476211638](https://doi.org/10.1023/A:1022476211638).
- [13] HAGLUND J. The q, t -Catalan Numbers and the Space of Diagonal Harmonics: vol. 41[M]. American Mathematical Society, 2008.
- [14] CHANDY K M, LAMPORT L. Distributed Snapshots: Determining Global States of Distributed Systems[J/OL]. ACM Transactions on Computer Systems, 1985, 3(1): 63-75. DOI: [10.1145/214451.214456](https://doi.org/10.1145/214451.214456).